

HAKILI LAB – Gestion Comptable

Dossier de passation : comprendre, faire tourner et faire évoluer l'application

Projet de Ouedraogo Pengdewende Afiya Imane

8 octobre 2026

Sommaire

Avant de commencer.....	3
1. Le projet : pourquoi cette application existe.....	5
2. La comptabilité qu'il faut connaître (pour développeur).....	8
3. L'application, écran par écran.....	13
4. Les modèles de saisie et les écritures qu'ils produisent.....	18
5. Architecture du code.....	23
6. La base de données PostgreSQL.....	27
7. Carte des fichiers du projet.....	33
8. Le tableau de bord.....	37
9. L'assistant IA.....	40
10. L'export vers Sage 100.....	43
11. Le serveur : se connecter, déployer, sauvegarder.....	46
12. Développer sur votre poste.....	51
13. État au 8 octobre 2026 et chantiers à mener.....	54
14. Historique et décisions.....	58
15. Glossaire et questions fréquentes.....	61

Avant de commencer

Rédigé le 8 octobre 2026, à la reprise du projet par une nouvelle développeuse ou un nouveau développeur.

Ce dossier explique tout le projet à quelqu'un qui n'en a jamais entendu parler : pourquoi il existe, comment s'en sert Hakili Lab, la comptabilité qu'il faut comprendre, le code fichier par fichier, la base de données, le serveur et ce qui reste à faire. Une fois lu, vous n'aurez plus besoin de personne pour continuer.

Par où commencer

Lisez dans l'ordre. Comptez une journée pour tout lire et une deuxième pour faire tourner l'application chez vous.

Ordre	Chapitre	Ce que vous y apprenez
1	Chapitre 1	Hakili Lab, ses centres, le problème que l'application résout
2	Chapitre 2	La comptabilité minimale à connaître (partie double, comptes, journaux, Sage)
3	Chapitre 3	Chaque écran, vu par la caissière, le valideur et le comptable
4	Chapitre 4	Les formulaires et les écritures exactes qu'ils produisent
5	Chapitre 5	Comment le code est organisé et comment Shiny rafraîchit les écrans
6	Chapitre 6	Les tables, les triggers, les fonctions SQL, les migrations
7	Chapitre 7	À quoi sert chaque fichier du dossier, y compris ceux qu'il ne faut pas toucher
8	Chapitre 8	Le tableau de bord du comptable et de la direction
9	Chapitre 9	L'assistant qui répond aux questions en français
10	Chapitre 10	Comment les écritures partent dans Sage 100, et les pièges de Sage
11	Chapitre 11	Le serveur, s'y connecter, déployer, sauvegarder, restaurer
12	Chapitre 12	Installer le projet sur votre poste, lancer les tests, Git
13	Chapitre 13	Où en est le projet le 08/10/2026 et ce qu'il faut faire ensuite, par priorité
14	Chapitre 14	Les décisions prises en cours de route et pourquoi
15	Chapitre 15	Les mots du métier et les questions qu'on se pose la première semaine

Le même contenu existe sous forme de fichiers séparés dans le dossier docs/ du dépôt, à garder à jour avec le code.

Ce qu'il faut retenir si vous ne lisez qu'une page

- **L'application** remplace les cahiers de caisse Excel (les « brouillards ») des cinq centres de Hakili Lab. Les caissières saisissent des opérations simples (« un élève paie ses frais »), l'application écrit l'écriture comptable, le comptable la valide, puis l'exporte dans **Sage 100**, qui reste le logiciel comptable officiel.
- **La technique** : Python, Shiny for Python pour l'interface, PostgreSQL pour les données, Docker sur un serveur Linux. L'assistant IA utilise l'API d'Anthropic.
- **Le serveur** : 167.233.234.219, compte imane, dossier ~/hakili_gestion_caisse, application sur le port 8022. Le compte n'a pas les droits administrateur : tout passe par Docker.

- **La règle d'or en production** : on déploie du code, jamais des données. Ne lancez jamais `docker compose down -v`, ni `schema.sql`, `seed.sql`, `production_reset.sql` ou `purge_avant_mise_en_service.sql` sur la base de production.
- **L'état au 08/10/2026** : les correctifs de l'audit du 07/10 sont écrits et testés (301 tests) mais **ni commités ni déployés**. Le tableau de bord est sur la branche `tableau-bord-v2`, pas encore fusionnée dans `main`. Voir chapitre 13.

À compléter par Afiya avant la remise du dossier

Certaines informations ne doivent pas être écrites dans un fichier, ou n'étaient pas connues au moment de la rédaction. Afiya les remet **en main propre** (jamais par e-mail ni dans le dépôt).

Information	Où elle sert	Statut
Mot de passe SSH du compte <code>imane</code> (ou mieux : un compte propre au repreneur, créé par l'administrateur)	Se connecter au serveur	À remettre en main propre
Contact de l'administrateur du serveur (nom, téléphone)	HTTPS, nginx, sudo, création de compte	À compléter :
Adresse web à laquelle les caissières ouvrent l'application	Tests en production, support	À compléter :
Fichier <code>.env</code> du serveur (mot de passe PostgreSQL, clé Anthropic)	Il reste sur le serveur, on ne le copie pas	Expliquer où il est, ne pas l'envoyer
Code d'accès actuel du compte <code>comptable</code> en production	Administrer l'application	À remettre en main propre, puis à changer
Compte GitHub : ajout du repreneur comme collaborateur sur <code>imaneodg/hakili-lab-comptabilite</code>	Pousser le code	À faire sur GitHub
Compte Anthropic qui porte la clé de l'assistant (qui paie, qui peut régénérer la clé)	Assistant IA	À compléter :
Emplacement du dossier Sage 100 et du format d'import <code>HAKILI_ECRITURES</code>	Export vers Sage	À compléter :
Nom et contact du comptable de Hakili Lab	Toute question de classement comptable	À compléter :

1. Le projet : pourquoi cette application existe

Hakili Lab

Hakili Lab est une entreprise de soutien scolaire installée à Ouagadougou (Burkina Faso). Elle propose surtout des **cours d'appui** (maths, physique-chimie, anglais) aux élèves, payés au mois par les parents, et organise en été un **camp de vacances**. Elle vend aussi quelques documents (attestations, bulletins) et des t-shirts.

L'activité est répartie sur **cinq centres**, chacun avec ses élèves, ses enseignants, ses dépenses et **sa propre caisse** :

Centre	Code dans l'application	Section analytique Sage	Ordre de numérotation
SIAO	SIA	SIAO	1
Tampouy	TAM	TAMP	2
Saaba	SAA	SAAB	3
Pissy	PIS	PSSY	4
Nagrin	NAG	NAGR	5
Siège	SIE	SIEG	6

Le Siège n'est pas un centre. Il n'a ni caisse ni élèves. Il existe seulement pour que le comptable puisse se connecter (l'écran de connexion demande un centre). Depuis le 08/10/2026, quand le comptable saisit une opération, il doit choisir le centre réel concerné.

Le SIAO a un rôle particulier. C'est le plus grand centre, celui qui a le plus de dépenses. Sur décision du propriétaire, chacun des quatre autres centres lui envoie chaque mois une somme (50 000 F au moment de la rédaction), appelée **contribution au SIAO** (ou « contribution au siège »). Cette somme n'est jamais remboursée. Il arrive aussi qu'un centre **prête** de l'argent au SIAO, ou que le SIAO paie des impôts pour le compte d'un centre. Ces mouvements entre centres ont leur propre formulaire (« Transfert entre centres ») et un motif obligatoire.

Chaque centre a **deux caisses physiques** :

- la **caisse principale (CP)**, où arrivent les paiements des élèves ;
- la **caisse menues dépenses (CMD)**, alimentée par la caisse principale, d'où sortent les petites dépenses (carburant, impressions, eau, vacances...).

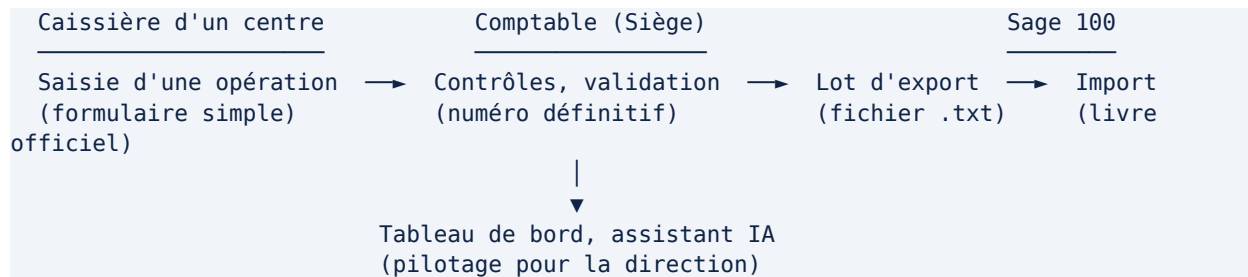
Il y a aussi **une seule banque**, commune à tous les centres (CBI, Coris Bank International). L'ancienne banque, BDU-BF, a été fermée en septembre 2026 ; son journal est conservé en lecture seule pour l'historique.

Ce qui se passait avant

Chaque caissière tenait un **brouillard de caisse** dans Excel : une feuille par mois, une ligne par opération, avec un libellé écrit à sa façon. En fin de période, le comptable reprenait ces fichiers, devinait les comptes, corrigeait les erreurs et ressaisissait tout dans **Sage 100**, le logiciel comptable officiel.

Les brouillards 2026 de Tampouy et Saaba (encore dans le dossier du projet) montrent ce que ça donnait : dates fausses (2006 au lieu de 2026), pages manquantes, soldes reportés qui ne collent pas, un même élève écrit de trois façons, le camp confondu avec les cours, un prêt au SIAO compté comme une charge. Le comptable passait son temps à rattraper, et la direction n'avait aucun chiffre fiable en cours d'année.

Ce que fait l'application



1. **Saisie** : la caissière choisit un modèle d'opération (« Encaissement de frais de scolarité », « Dépense courante »...), remplit trois ou quatre champs, et voit l'écriture comptable se construire en direct. Elle n'a jamais à taper « débit » ou « crédit ».
2. **Contrôle** : l'application vérifie l'équilibre, les comptes, les tiers, les soldes de caisse, et signale les anomalies.
3. **Validation** : le comptable (ou un validateur local pour son centre) valide les pièces. Chaque pièce reçoit alors un **numéro définitif** sans trou, ou est renvoyée au centre avec un motif.
4. **Export** : le comptable crée un **lot d'export** ; l'application produit le fichier que Sage sait importer, et fige les pièces qu'il contient.
5. **Pilotage** : le comptable dispose d'un **tableau de bord** par centre (résultat, élèves payants, caisse, fiabilité) et d'un **assistant IA** à qui l'on pose des questions comme « combien avons-nous reçu ce mois-ci ? ».

Sage reste le livre officiel. L'application prépare et contrôle les écritures ; elle ne produit ni bilan ni liasse fiscale.

Les personnes

Qui	Rôle dans l'application	Ce qu'il fait
Caissière ou caissier d'un centre	saisie , rattaché à son centre	Saisit les opérations de son centre, corrige les pièces renvoyées
Validateur local (rare)	validation , rattaché à un centre réel	Valide les pièces de son seul centre
Comptable	validation , rattaché au Siège (SIE)	Voit tout, valide, exporte vers Sage, gère le référentiel et les utilisateurs, consulte tableau de bord et assistant
Directeur	Pas encore de rôle propre	Lit le tableau de bord avec le comptable ; un accès direct est prévu plus tard

Décision prise : le rôle validation ne doit pas être donné aux centres. En pratique, seul le comptable valide.

Ce que le projet n'est pas

- Ce n'est pas un logiciel comptable complet : pas de bilan, pas de lettrage, pas de déclaration fiscale. Tout cela se fait dans Sage.
- Ce n'est pas un outil générique : les modèles de saisie, les comptes, les libellés et les règles du tableau de bord sont taillés pour Hakili Lab. Avant d'ajouter une fonction « standard », vérifiez qu'elle correspond à la façon dont les centres travaillent vraiment.

- Les brouillards Excel historiques **ne sont pas importés en production** (décision du 16/09/2026). La base de production démarre au 01/04/2026 avec la saisie réelle. Les brouillards 2026 ont seulement servi à construire une base de démonstration locale pour tester le tableau de bord.

Les exigences de départ

Elles viennent des instructions du projet et restent valables :

- un outil **réel et durable**, pensé pour les centres, les journaux, le plan comptable et le circuit de validation de Hakili Lab ;
- un code de niveau professionnel, fiable, découpé par domaine (saisie, validation, export, référentiel, assistant) ;
- une interface **sobre** : beaucoup de fonctions, mais le détail se déplie (onglets, accordéons, « afficher plus ») au lieu d'encombrer l'écran ;
- une cohérence visuelle et fonctionnelle entre tous les modules.

Afiya a aussi fixé deux règles de travail qu'il est sage de garder : un changement de présentation ne touche jamais la logique métier, les identifiants des champs ni les appels serveur ; et le code reste simple et lisible plutôt qu'astucieux.

2. La comptabilité qu'il faut connaître (pour développeur)

Vous n'avez pas besoin d'être comptable pour maintenir cette application. Il faut en revanche comprendre une dizaine de notions, parce que le code les applique partout et que les tests vérifient qu'elles sont respectées. Ce chapitre les explique avec les exemples réels de Hakili Lab.

2.1 La partie double

Chaque opération s'écrit sur **au moins deux comptes**. Un compte reçoit (on écrit le montant au **débit**), un autre donne (on écrit le montant au **crédit**). La somme des débits est **toujours** égale à la somme des crédits. C'est la règle de base, et la base de données la vérifie elle-même (trigger `trg_equilibre_piece`).

Exemple : un élève paie 15 000 F de frais d'octobre en espèces à la caisse principale de Pissy.

Compte	Intitulé	Débit	Crédit
571100	Caisse principale	15 000	
411000 (tiers 411KABORE)	Clients (élèves)		15 000

La caisse **augmente** (débit), et la dette de l'élève envers Hakili Lab est soldée (crédit sur son compte client).

Moyen mnémotechnique côté trésorerie : **l'argent qui entre en caisse est au débit de la caisse, l'argent qui sort est au crédit de la caisse.**

Dans la base, une ligne a soit un débit, soit un crédit, jamais les deux, et jamais négatif (contrainte `ck_montants_positifs`).

2.2 Pièce, ligne, écriture

- Une **ligne** = un compte, un montant au débit ou au crédit, un libellé. Table `ecritures`, une ligne par enregistrement.
- Une **pièce** = l'ensemble des lignes d'une même opération (colonne `id_piece`). Elle doit être équilibrée.
- Deux pièces peuvent être **liées** (colonne `id_lien`) : c'est le cas quand une opération touche deux journaux, par exemple l'approvisionnement de la petite caisse (une pièce dans CP, une dans CMD). Elles sont validées, exportées et supprimées ensemble.

2.3 Le plan comptable SYSCOHADA

Le Burkina Faso applique le référentiel **SYSCOHADA** (système comptable de l'OHADA). Les comptes ont 6 chiffres chez Hakili Lab ; le premier chiffre est la **classe** :

Classe	Nature	Exemples chez Hakili Lab
1	Capitaux, emprunts	160100 Emprunts bancaires, 160200 Emprunts auprès des associés
2	Immobilisations	244100 Matériel et mobilier de bureau, 272800 Prêts au personnel
4	Tiers (clients, fournisseurs, personnel, État)	411000 Clients (élèves), 401000 Fournisseurs, 422000 Personnel, 447810 Retenues

Classe	Nature	Exemples chez Hakili Lab
5	Trésorerie	571100 Caisse principale, 571200 Caisse menues dépenses, 521200 Banque CBI, 585000 Virements de fonds
6	Charges	605300 Carburant, 622200 Loyer, 632710 Vacances, 646200 Timbres
7	Produits	706110 Cours d'appui maths/PC, 707810 Frais de document

Le plan complet (1 040 comptes) a été chargé en production le 18/09/2026 depuis l'export Sage (Plan_comptable.xlsx). Le fichier sql/seed.sql n'en contient qu'une soixantaine, pour une base de développement.

Dans la table comptes, chaque compte a une **nature** (charge, produit, tresorerie, bilan, tiers), un indicateur tiers_obligatoire (oui pour 401000, 411000, 422000) et un indicateur depense_courante (les comptes proposés dans le formulaire « Dépense courante »).

2.4 Les tiers et les comptes collectifs

Un **tiers** est une personne ou une entreprise avec qui Hakili Lab a un compte : un élève, un fournisseur, un enseignant. Plutôt que de créer un compte de classe 4 par élève, la comptabilité utilise un **compte collectif** (411000 pour tous les élèves) et un **code tiers** qui précise qui (411KABORE).

Convention de Hakili Lab, respectée par les 1 454 tiers du plan Sage : **le code tiers commence par les trois chiffres de son collectif**, suivis du nom en majuscules sans espace ni accent.

Collectif	Type	Exemple de code
411000	élève (client)	4110UEDRAOGOAFIYA
401000	fournisseur, vacataire, bailleur	401SONABEL, 401BAILLEURSAABA
422000	personnel salarié	422KABORE

Longueur maximale : 17 caractères. Le dossier Sage est réglé sur 17 et tronque sans prévenir au-delà ; deux élèves différents finiraient sur le même compte. L'application refuse tout code plus long (fonctions ajouter_tiers, resoudre_tiers dans logic/donnees.py).

Quand une caissière tape un nom d'élève absent de la liste, l'application **crée le tiers au moment de l'enregistrement** (resoudre_tiers). Conséquence importante pour l'export : ce tiers n'existe pas encore dans Sage. Il faut le créer dans Sage avant d'importer, sinon la ligne est rejetée (voir chapitre 10).

Chaque année, à la rentrée, le comptable clique sur « Nouvelle année académique » : les tiers de l'année passée disparaissent des listes de saisie (actif_annee = 'non') sans être supprimés. Un élève qui revient est réactivé avec son ancien code dès qu'on retape son nom.

2.5 Les journaux

Un **journal** regroupe les pièces d'une même nature. Sage impose de travailler journal par journal.

Journal	Intitulé	Compte de trésorerie	Type	Remarque
CP	Caisse principale	571100	trésorerie	une caisse physique par centre
CMD	Caisse menues dépenses	571200	trésorerie	une caisse physique par centre

Journal	Intitulé	Compte de trésorerie	Type	Remarque
Banque	CBI	521200 en production	trésorerie	compte unique, commun à tous les centres
BDU - BF	Ancienne banque	521100	trésorerie	inactif depuis le 11/09/2026, historique seulement
VTE	Ventes	aucun	opérations	factures de cours d'appui
ACH	Achats	aucun	opérations	factures fournisseurs, états de vacation

Attention : sql/seed.sql décrit encore le journal Banque sur 521100. La production et Sage sont sur **521200**. Sans conséquence tant qu'on ne recrée pas une base de zéro pour la production ; à corriger dans le seed (voir chantiers).

Un journal de **trésorerie** a un compte de contrepartie fixe : dans toute pièce de ce journal, l'application écrit elle-même la ligne de caisse ou de banque. Un journal d'**opérations** (VTE, ACH) n'a pas de caisse : ses modèles écrivent leurs deux comptes.

2.6 Les caisses physiques et les soldes

Chaque centre a sa propre CP et sa propre CMD. Le solde d'une caisse se calcule par centre :

```
solde = solde d'ouverture du centre (table soldes_ouverture_centre)
+ somme des débits du compte de caisse pour ce centre
- somme des crédits du compte de caisse pour ce centre
```

La banque est unique : son solde d'ouverture est dans journaux.solde_ouverture et ses mouvements ne sont jamais ventilés par centre.

Un solde de caisse **négatif** est impossible dans la réalité (on ne peut pas sortir plus d'espèces qu'on n'en a). L'onglet Contrôles le signale : soit une pièce manque, soit un montant est faux, soit le solde d'ouverture n'a pas été saisi.

2.7 Le compte 585000 « Virements de fonds »

Quand de l'argent passe d'une caisse à une autre **sans quitter Hakili Lab**, ce n'est ni une recette ni une dépense. SYSCOHADA fait passer ce mouvement par un **compte de passage**, le 585000 : la caisse de départ le crédite, la caisse d'arrivée le débite, et le 585000 revient à zéro quand les deux côtés sont saisis.

Trois cas dans l'application :

Opération	Côté départ	Côté arrivée	Saisie
Approvisionnement de la CMD	CP crédit 571100, débit 585000	CMD débit 571200, crédit 585000	une seule saisie, deux pièces liées
Versement d'espèces en banque	CP crédit 571100 (+ timbre), débit 585000	Banque débit 521200, crédit 585000	une seule saisie, deux pièces liées
Transfert entre centres (contribution, prêt...)	caisse du centre donateur	caisse du centre destinataire	chaque centre saisit sa face , rapprochées par une référence TRF - AAAAMM - 0001

Pour le transfert entre centres, chaque face est saisie par son propre centre parce que l'argent met parfois des semaines à arriver et qu'un centre ne doit jamais écrire dans les livres d'un autre. Le rapprochement se fait par la référence ; un écart de montant reste visible.

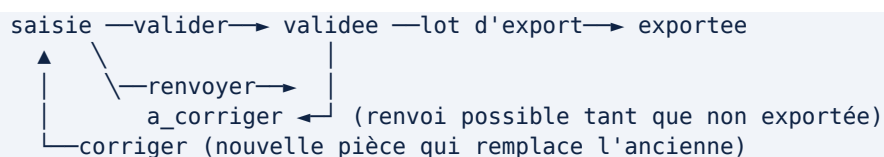
Les tableaux de bord excluent ces virements des recettes et des dépenses, **sauf** quand le motif en fait une charge pour le centre donateur (contribution au SIAO) ou un mouvement hors exploitation (prêt, impôt).

2.8 Le compte 471000 « Compte d'attente »

Quand une caissière ne sait pas sur quel compte passer une opération, elle peut utiliser le formulaire « Écriture libre » avec le **compte d'attente 471000**. C'est un compte provisoire, interne à l'application : il **n'existe pas dans Sage** et ne doit jamais y partir.

- Une pièce sur 471000 **ne peut pas être validée** (contrôle bloquant).
- Le comptable la **reclasse** depuis l'onglet Contrôles : il choisit le bon compte (et le tiers si besoin), la pièce repart en attente de validation.

2.9 Statuts d'une pièce et numérotation



- **Numéro provisoire** à la saisie : {centre} - {AAMM} - {nnn}, par centre et par mois (ex. PIS-2610-012).
- **Numéro définitif** à la validation : {préfixe du journal} {AAMM} {nnn}, par journal et par mois, **tous centres confondus** (ex. CP2610003). Dans un lot validé, les numéros suivent la **date**, puis l'**ordre des centres** (SIAO, Tampouy, Saaba, Pissy, Nagrin), puis l'heure de saisie. Une pièce oubliée et validée plus tard prend le numéro suivant, comme dans Sage.
- Une pièce **renvoyée** après validation garde son numéro de côté (num_reserve) et le retrouve à la revalidation : pas de trou.
- La numérotation continue sans trou est une obligation de l'Acte uniforme OHADA. La base refuse deux pièces avec le même numéro dans un journal (trigger trg_numero_unique), et la validation verrouille les lignes pour que deux validateurs simultanés ne perdent pas de numéros.

2.10 Une pièce exportée ne se modifie plus : l'extourne

Une fois dans Sage, une pièce fait partie du livre officiel. On ne la corrige plus en la modifiant : on passe une **écriture d'extourne** (la même écriture à l'envers, qui annule la première), puis la bonne écriture. Dans l'application, une pièce exportée ne peut être ni modifiée, ni supprimée, ni renvoyée ; la base l'interdit aussi (trigger trg_proteger_ecritures).

De même, un **mois clôturé** par le comptable ne reçoit plus aucune pièce et ses pièces ne changent plus. On ne clôture qu'un mois entièrement exporté.

2.11 Base caisse ou base engagement

Deux façons de compter le résultat d'un mois :

- **base caisse** : on compte l'argent à la date où il entre ou sort de la caisse. Un élève qui paie en mars ses cours d'avril compte en mars.
- **base engagement** (ou « rattachement ») : on compte le cours dans le mois où il est donné. Les frais d'avril payés en mars comptent en avril.

Le tableau de bord est en base caisse (décision du 02/10/2026, comme le fichier d'analyse de Tampouy qui a servi de modèle). L'assistant IA sait faire les deux, mais par défaut il donne les chiffres du tableau de bord pour « bénéfice, résultat, marge, charges », pour que la direction n'ait jamais deux réponses différentes.

2.12 Exploitation et hors exploitation

Le tableau de bord sépare :

- **les encaissements d'exploitation** : cours d'appui, camp de vacances, frais de document, contributions reçues (pour le SIAO) ;
- **les charges d'exploitation** : vacations et salaires, contribution au SIAO, loyer, eau, électricité, internet, fournitures, entretien, frais bancaires...
- **les sorties hors exploitation** : remboursements d'emprunts, prêts entre centres, achats d'équipement, avances au personnel, dossiers de conseil et, pour l'instant, les **impôts et taxes**.

Résultat = encaissements d'exploitation – charges d'exploitation. Les sorties hors exploitation sont affichées à part.

Point en suspens : en SYSCOHADA, les impôts et taxes (comptes 64) sont des charges d'exploitation. Les passer en exploitation est une ligne de SQL (`UPDATE rubriques_tableau SET bloc = 'charge' WHERE rubrique = 'impots' ;`), mais la décision a été **reportée** par Afiya le 08/10/2026. À trancher avec la direction.

2.13 Retenues à la source

Sur certaines factures, Hakili Lab retient une partie du montant pour la reverser à l'État :

- **état de vacation** : retenue de 2 % automatique (compte 447810) ;
- **gardiennage** : retenue de 5 % (447820) ;
- **loyer** : retenue IRF (447800), montant saisi à la main car le taux varie d'une pièce à l'autre.

Le fournisseur reçoit le net ; la charge est comptée en brut.

2.14 Sage 100 en deux phrases

Sage 100 Comptabilité i7 est le logiciel comptable officiel de Hakili Lab. L'application lui envoie un fichier texte (un lot d'export) qu'on importe par un **format paramétrable** nommé HAKILI_ECRITURES ; tout le détail, y compris les pièges de longueur des codes, est dans chapitre 10.

3. L'application, écran par écran

Ce chapitre décrit ce que voit chaque utilisateur. Faites ce tour vous-même sur votre base locale (chapitre 12) avant de lire le code : la moitié des choix techniques s'expliquent en voyant l'écran.

3.1 La connexion

L'écran de connexion demande trois choses : le **centre**, le **nom d'utilisateur** et le **code personnel**.

- Un identifiant appartient à une personne, pas à un centre : c'est ce qui permet de savoir qui a saisi et qui a validé (colonnes `saisi_par`, `valide_par`).
- Le nom d'utilisateur n'est pas sensible à la casse.
- Le code est haché avec **bcrypt** en base. Six caractères au minimum ; les codes trop faciles (123456, 000000...) sont refusés.
- Après **5 essais faux**, le compte est bloqué **15 minutes**.
- Un code **provisoire** donné par le comptable doit être changé à la connexion : une fenêtre s'ouvre et la session ne démarre qu'une fois le nouveau code enregistré.
- Le navigateur retient le dernier centre choisi sur ce poste (stockage local).
- Si le comptable désactive un compte pendant qu'il est connecté, la session se ferme d'elle-même dans les deux secondes.

Après connexion, la barre latérale à gauche affiche le logo, les onglets autorisés, puis en bas le centre, le nom et le rôle, avec « Fermer la session ».

3.2 Qui voit quoi

Onglet	Caissière (saisie)	Valideur local (validation , centre réel)	Comptable (validation , Siège)
Saisie	oui, pour son centre	oui, pour son centre	oui, pour le centre qu'il choisit
Brouillard	son centre	son centre	tous les centres
Validation	non	son centre	tous les centres, filtre par centre
Export Sage	non	consultation de son centre	création des lots, réimport
Contrôles	son centre, sans la banque	son centre, sans la banque	tout, banque comprise
Tableau de bord	non	non	oui
Assistant IA	non	non	oui
Référentiel	création de comptes et de tiers, changement de son code	idem	tout : utilisateurs, soldes d'ouverture, nouvelle année, clôtures

Ces droits sont **vérifiés côté serveur** à chaque action, pas seulement par l'absence d'un bouton. C'est un point sur lequel le projet a été audité plusieurs fois (voir chapitre 14) : masquer un widget ne protège rien, un client peut envoyer n'importe quelle valeur d'input à Shiny.

3.3 Saisie

L'écran a deux cartes côte à côte.

À gauche, « Informations sur l'opération » :

1. le **modèle d'opération**, regroupé par journal (CP, CMD, Banque, VTE, ACH, puis « Autres ») ;
2. la **date** de l'opération (refusée si elle dépasse aujourd'hui de plus de 7 jours, ou si le mois est clôturé) ;
3. pour le comptable seulement : le **centre concerné**, obligatoire ;
4. le journal, affiché en texte (il est imposé par le modèle ; seul « Écriture libre » le laisse choisir) ;
5. les champs propres au modèle (élève, montant, répartition par mois, nature de la dépense, etc.).

À droite, « **Écriture générée** » : l'écriture comptable telle qu'elle sera enregistrée, construite en direct, avec un ruban vert si elle est équilibrée et complète, ou un message qui dit ce qui manque. Puis les boutons **Enregistrer la pièce** et **Vider**.

Le bouton Enregistrer est désactivé pendant l'enregistrement, pour qu'un double-clic ne crée pas deux pièces.

Détails qui comptent pour la caissière :

- les champs « élève », « fournisseur » ou « bénéficiaire » proposent la liste des tiers existants et acceptent un nom nouveau, qui deviendra un tiers à l'enregistrement ;
- le tableau de **répartition** (encaissement, règlement fournisseur) permet de couvrir plusieurs mois en un paiement : une ligne par nature (Frais / Avance / Solde, ou Paiement / Avance / Solde), avec un ou plusieurs mois et un montant. L'écran affiche ce qui reste à répartir. Pour une avance, l'application propose le mois qui suit le dernier mois déjà payé par ce tiers ;
- le choix **Activité** (Cours d'appui / Camp de vacances) reste d'une pièce à l'autre : pendant le camp, la caissière n'a pas à le rechoisir à chaque fois ;
- les montants ne sont envoyés au serveur qu'en quittant le champ, pour que l'écriture ne se reconstruise pas à chaque chiffre tapé.

Le détail de chaque modèle et des écritures produites est au chapitre 4.

3.4 Brouillard

La liste des pièces, avec trois filtres : statut, journal, période (du 1er du mois à aujourd'hui par défaut). Une période qui commence avant le 1er janvier de l'année en cours va relire l'historique en base.

Au-dessus du tableau, un bandeau d'indicateurs : pièces affichées, entrées en caisse, sorties de caisse, virements internes (les mouvements via 585000, comptés une seule fois), puis le solde de chaque caisse visible. Les soldes portent toujours sur **tout l'historique**, quel que soit le filtre.

Actions sur la pièce sélectionnée :

- **Corriger la pièce** : rouvre la pièce dans Saisie, avec toutes ses valeurs d'origine (elles sont gardées dans `valeurs_json`). À l'enregistrement, la nouvelle pièce remplace l'ancienne dans la même transaction. Possible tant que la pièce n'est pas validée. Un bandeau rappelle qu'une correction est en cours, avec « Annuler la correction ».
- **Supprimer** : seulement une pièce non validée. Le contenu est archivé dans `suppressions_ecritures` (qui, quand, quoi) avant suppression.

Une pièce liée (approvisionnement, versement en banque) est toujours supprimée ou corrigée avec sa jumelle.

3.5 Validation

La file des pièces au statut « en attente de validation » ou « à corriger ». Le comptable peut filtrer par centre.

- **Tout sélectionner / Tout désélectionner** : ne prend que les lignes affichées (filtres compris). La grille ne garde qu'une ligne sur un clic simple ; Ctrl+clic ajoute, Maj+clic prend une plage. Le compteur à droite des boutons dit combien de pièces sont réellement choisies.
- **Valider les pièces choisies** : contrôle d'abord les anomalies bloquantes (une fenêtre les liste si besoin), puis attribue les numéros définitifs. Le message dit ce qui a été validé **et** ce qui a été écarté, avec la raison (pièce à corriger, compte d'attente à reclasser, déjà validée...).
- **Renvoyer pour correction** : motif obligatoire. La pièce repasse « à corriger » dans le brouillard du centre. Une pièce déjà exportée ne peut pas être renvoyée.
- **Corriger la pièce** : le validateur corrige lui-même au lieu de renvoyer.

3.6 Export Sage

Réservé au comptable pour la création. L'écran a deux cartes.

« **Pièces validées à exporter** » : une période (du... au...) qui part par défaut de la plus ancienne pièce validée non exportée, et un filtre journal. Le bandeau donne le nombre de pièces, de lignes et le total au débit, et prévient si des pièces de la période sont déjà dans Sage (elles n'entrent jamais dans un nouveau lot) ou si des pièces validées sont hors période. Le bouton **Créer le lot d'export** demande confirmation, puis fige d'un coup la liste des pièces, le fichier Sage et leur passage au statut « exportée ».

« **Lots déjà créés** » : choix d'un lot, téléchargement du **fichier Sage (.txt)** tel qu'il a été créé, ou d'une **version Excel** pour relire. Le fichier d'un lot se retélécharge à l'identique à tout moment.

Replié en bas : « **Réexporter des pièces déjà envoyées à Sage** », pour le seul cas où ces pièces ne sont plus dans Sage (dossier recréé ou restauré). Motif obligatoire, fichier `sage_REIMPORT_<du>_au_<au>.txt`, trace dans la table `reimports_sage`. Les statuts ne changent pas.

Le protocole complet d'import dans Sage est au chapitre 10.

3.7 Contrôles

Le tableau des anomalies sur les pièces **non encore exportées** (une pièce exportée se corrige par extourne, pas ici). Deux gravités : **bloquante** (empêche la validation) et **à vérifier**.

Anomalie	Gravité
Pièce déséquilibrée	bloquante
Pièce de caisse ou de banque sans ligne de trésorerie	bloquante
Compte absent du plan comptable	bloquante
Compte à tiers obligatoire sans code tiers (le lettrage serait impossible dans Sage)	bloquante
Même compte débité et crédité pour le même tiers (la pièce n'a aucun effet)	bloquante
Tiers inconnu du référentiel	bloquante

Anomalie	Gravité
Code tiers de plus de 17 caractères	bloquante
Compte d'attente 471000 sur une pièce non validée	bloquante
Approvisionnement sans sa pièce jumelle	bloquante
Numéro définitif incohérent avec le journal	à vérifier
Ligne sans libellé	à vérifier
Solde de caisse négatif (par centre), ou de banque pour le comptable	à vérifier

En sélectionnant une pièce sur 471000, un panneau « **Attribuer le compte et le tiers** » permet de la reclasser ; elle retourne dans la file de validation.

Si des pièces citent un tiers absent du référentiel (cas d'une restauration partielle), le comptable voit un bouton pour recréer les fiches manquantes.

3.8 Tableau de bord (comptable)

En haut : la période et les centres. Deux sous-pages : **Vue d'ensemble** (trois chiffres par centre, alertes, comparatif des centres, compte de résultat mensuel) et **Fiche centre** (Résultat, Élèves, Caisse, Fiabilité). Les cibles de marge et de trésorerie se règlent par centre. Détails au chapitre 8.

3.9 Assistant IA (comptable)

Une conversation : on pose une question en français (« combien saab a encaissé en fevrier », « fais-moi le point »), l'assistant répond en deux phrases avec le chiffre, et affiche sous la réponse des cartes dépliantes avec le détail ou un graphique. Six questions essentielles sont proposées en pastilles à l'ouverture, et un menu « Questions fréquentes » en propose d'autres. Détails au chapitre 9.

3.10 Référentiel

- **Plan de comptes et Comptes de tiers** : listes complètes (repliées à 8 lignes, filtrables), bouton « + » pour créer. La création est ouverte à tous les rôles depuis le 23/09/2026, pour qu'une caissière ne soit pas bloquée par un tiers manquant ; le format est vérifié (compte sur 6 chiffres, code tiers préfixé par son collectif, 17 caractères au plus).
- **Utilisateurs** (comptable) : créer (identifiant, nom, rôle, centre, code de 6 caractères au moins), désactiver (rien n'est supprimé), réinitialiser un code oublié avec un code provisoire.
- **Paramètres** (comptable) :
 - **Soldes d'ouverture des caisses** : un montant par centre pour CP et CMD, un seul pour la banque. À saisir à la mise en service, sinon les soldes sont faux et souvent négatifs ;
 - **Nouvelle année académique** : une fois par an, à la rentrée ;
 - **Clôture des mois** : clôturer ou rouvrir un mois. Refusé s'il reste des pièces non exportées dans le mois.
- **Mon code d'accès** (tout le monde) : changer son code en donnant l'ancien.

3.11 Le rythme normal d'un mois

1. Chaque jour, les caissières saisissent leurs opérations.

2. Plusieurs fois par semaine, le comptable regarde Contrôles, reclasse les 471000, puis valide.
3. En début de mois, il saisit ou vérifie les transferts (contribution de chaque centre au SIAO, avec le bon motif des deux côtés).
4. Une fois le mois validé, il crée le lot d'export, l'importe dans Sage, vérifie les totaux.
5. Quand Sage est à jour, il clôture le mois dans le Référentiel.
6. Il présente le tableau de bord à la direction.

4. Les modèles de saisie et les écritures qu'ils produisent

Tout est dans `logic/modeles.py`, dans la liste `MODELES`. Un modèle décrit une opération **telle que la caisse la vit** (« un élève paie ses frais »), et sa fonction `lignes` la traduit en écriture comptable. C'est le cœur métier de l'application : lisez ce fichier en entier, il est très commenté.

4.1 Comment un modèle est construit

```
{
    "id": "depense",                # identifiant stable, enregistré dans
    ecritures.modele
    "titre": "Depense courante",    # texte du menu
    "aide": "Carburant, impression...", # sous-titre
    "journal": "CMD",              # journal imposé (None = au choix)
    "champs": [ {...}, {...} ],    # les questions posées
    "libelle": lambda v: ...,       # le libellé de la pièce
    "lignes": lambda v, cc: _df(...), # les lignes d'écriture ; cc = compte de caisse
    du journal
}
```

Types de champ ("t") : tiers, compte, libelle, texte, montant, mois, choix, centre, repartition. Options utiles :

- "pref" et "collectif" sur un champ tiers ("411", "411000") ;
- "filtre": "depense_courante" ou "choix": [...] sur un champ compte, pour limiter la liste ;
- "default" sur un montant (timbre à 50 F) ;
- "garde": True pour garder le choix d'une pièce à l'autre ;
- "lie": "CMD" : l'opération produit une seconde pièce dans ce journal, liée à la première ;
- "retire": True : le modèle disparaît du menu mais reste connu pour relire et corriger les anciennes pièces.

Ajouter un modèle : copier un bloc existant, changer l'id, les champs et la fonction `lignes`. L'interface (`app.py`, sorties `m_champs`, `m_aperçu`) se construit toute seule à partir de cette description. Ajoutez un test dans `tests/`.

Tous les libellés sont mis en majuscules et coupés à **60 caractères** (`LIBELLE_MAX`). Quand il faut couper, l'application coupe toujours le **nom** en premier, jamais la nature ni les mois : le code tiers de la ligne identifie déjà la personne.

4.2 Les modèles, un par un

Dans les tableaux, caisse désigne le compte de trésorerie du journal (571100 en CP, 571200 en CMD, 521200 en banque).

Encaissement de frais de scolarité (encaissement, journal CP)

Le formulaire le plus utilisé. Champs : élève (411), activité (Cours d'appui / Camp de vacances), montant total reçu, tableau de répartition (nature Frais / Avance / Solde, un ou plusieurs mois, montant ; 6 lignes au plus).

Compte	Libellé	Débit	Crédit
caisse	FRAIS CA - NOM	total	

Compte	Libellé	Débit	Crédit
411000 + tiers	FRAIS CA OCT - NOM		ligne 1
411000 + tiers	AVANCE CA NOV-DEC - NOM		ligne 2

Pour le camp : FRAIS CV, AVANCE CV, SOLDE CV. C'est ce libellé qui permet au tableau de bord de séparer le camp des cours. Les mois sont toujours écrits dans l'ordre de l'année scolaire (septembre → août), abrégés et tous cités (OCT-NOV-DEC). Si la somme des lignes ne fait pas le total, la pièce est déséquilibrée et ne s'enregistre pas.

Pourquoi « SOLDE » et pas « RATRAPAGE » : c'est le mot du comptable quand un mois partiellement payé est enfin réglé.

Frais de document (document, CP)

Champs : nom de l'élève (texte libre), montant.

Compte	Débit	Crédit
caisse	montant	
707810 Frais de dossier et de documents		montant

Approvisionnement de la CMD (approvisionnement, CP, lié à CMD)

Champ : montant. Une saisie, deux pièces liées. Libellé APPROV CMD.

Pièce	Compte	Débit	Crédit
CP	585000	montant	
CP	571100		montant
CMD	571200	montant	
CMD	585000		montant

Le sens est toujours CP → CMD (596 pièces dans l'historique, jamais l'inverse).

Versement d'espèces en banque (versement_banque, CP, lié à Banque)

Champs : montant versé, timbre (50 F par défaut, compte 646200).

Pièce	Compte	Débit	Crédit
CP	585000	montant	
CP	646200 (TIMBR/...)	timbre	
CP	571100		montant + timbre
Banque	521200	montant	
Banque	585000		montant

Transfert entre centres (transfert_interne, journal au choix : CP, CMD ou banque)

Champs : centre qui remet, centre qui reçoit (le SIAO par défaut), **motif obligatoire sans valeur par défaut**, montant, libellé facultatif.

Motifs (MOTIFS_TRANSFERT) :

Clé	Texte	Rubrique au tableau de bord
contribution	Contribution au fonctionnement du siège	charge d'exploitation du donateur, encaissement du SIAO
pret	Prêt (à rembourser)	hors exploitation
remboursement	Remboursement d'un prêt	hors exploitation
impot	Impôts payés par le siège pour le centre	impôts

Il n'y a pas de champ « sens » : le sens se déduit du centre de la personne connectée. Si elle est le donateur, elle enregistre une **sortie** (585000 au débit, caisse au crédit) et reçoit une nouvelle référence TRF - AAAAMM-nnnn. Si elle est le destinataire, elle enregistre une **entrée** (caisse au débit, 585000 au crédit), rattachée à la plus ancienne sortie qui l'attend (premier entré, premier sorti, sans regarder le montant, pour qu'un écart reste visible). Le destinataire doit choisir **le même motif** que le donateur, sinon l'enregistrement est refusé. Un centre ne peut pas saisir un transfert entre deux autres centres.

Le libellé commence par le mot du motif (CONTRIBUTION TAM>SIA, PRET APPROV SIAO) ; la caissière peut garder son propre vocabulaire derrière, le rapprochement ne lit jamais le libellé.

Dépense courante (depense, CMD)

Champs : nature (seuls les comptes marqués depense_courante = 'oui' : eau minérale, électricité, carburant, fournitures, petits équipements, maintenance, entretien, téléphone, internet, frais mobile money, impressions, réception...), libellé, montant.

Compte	Débit	Crédit
compte de charge choisi	montant	
caisse		montant

Règlement d'un fournisseur (fournisseur, CMD) et par banque (fournisseur_banque, Banque)

Champs : fournisseur (401), libellé, activité, montant payé, répartition (Paiement / Avance / Solde, mois, montant), timbre (en caisse seulement).

Compte	Libellé	Débit	Crédit
401000 + tiers	PAIEMENT ETAT VACATION OCT - NOM	ligne 1	
401000 + tiers	AVANCE LOYER NOV-DEC - NOM	ligne 2	
646200	TIMBR/...	timbre	
caisse ou banque			total + timbre

Pour le camp, le mot CAMP est ajouté en tête du libellé tapé. Une faiblesse connue : la nature de la charge n'est que dans le libellé libre, le tableau de bord la devine par mots-clés (voir chapitre 8 et chantiers).

Salaire ou vacation (remuneration, CMD)

Champs : bénéficiaire (422), activité, mois, montant. Libellé REMUNERATION OCTOBRE/NOM (ou REMUNERATION CAMP ...).

Compte	Débit	Crédit
422000 + tiers	montant	
caisse		montant

Frais bancaires ou règlement d'impôt (frais_bancaires, Banque)

Champs : nature parmi 631800 (frais bancaires), 447100, 447200, 447800, 447810 (impôts et retenues), libellé, montant. Écriture : compte choisi au débit, banque au crédit.

Facture de cours d'appui (facture, VTE)

Champs : élève, prestation (706110, 706120 ou 706130), numéro de facture, montant. Libellé repris de la forme utilisée depuis 2020 : FACT N*057/N0V/26 NOM.

Compte	Débit	Crédit
411000 + tiers	montant	
706xxx		montant

La facture crée la créance ; l'encaissement la solde plus tard.

État de vacation (vacation, ACH)

Champs : vacataire (401), mois, montant brut. Retenue de 2 % automatique.

Compte	Débit	Crédit
632710 Honoraires et vacations	brut	
447810 (RETENUE 2%/...)		2 %
401000 + tiers		brut - 2 %

Facture fournisseur (achat, ACH)

Champs : fournisseur, nature de la charge (loyer, gardiennage, eau, électricité, entretien, internet, formation, t-shirts, petit matériel, carburant, fournitures), libellé, montant brut, retenue (montant, 0 par défaut). Le compte de retenue se déduit de la charge : loyer → 447800, gardiennage → 447820, vacations → 447810.

Écriture libre (libre, journal au choix)

La soupape pour les cas qu'aucun modèle ne couvre. Les questions dépendent du type de journal :

- **journal de trésorerie** : sens (entrée ou sortie), autre compte (471000 si on ne sait pas), libellé, montant. La caisse est l'autre côté ;
- **journal d'opérations** (VTE, ACH) : sens, compte concerné, libellé, montant. **Le 471000 est toujours l'autre côté.** Ainsi toute écriture libre de ce type apparaît dans les Contrôles et ne peut pas inverser silencieusement le sens d'une opération.

Une note libre peut accompagner la pièce (observation).

Les comptes à tiers obligatoire sont exclus de l'écriture libre : elle n'a pas de champ tiers.

Avance de paiement multi-mois (avance_paiement_multimois) — retiré

Remplacé par la répartition du modèle Encaissement. Gardé avec "retire": True pour relire et corriger les anciennes pièces.

4.3 Le chemin d'une saisie dans le code

1. `app.py::valeurs()` rassemble les valeurs des champs (`ch_*`, `m_rep_*`).
2. `app.py::operation()` appelle `md.construire_operation(id_modele, valeurs, journal, journaux)`, qui :
 - trouve le compte de caisse du journal (`compte_contrepartie`) ;
 - normalise les valeurs (montants en float, répartition en liste) ;
 - compose le libellé ;
 - appelle la fonction `lignes` et, si le modèle a un "lie", recommence pour le second journal.
3. `m_aperçu()` et `m_ruban()` affichent le résultat ; `md.operation_equilibree()` et `md.comptes_annules()` disent s'il est enregistrable.
4. Au clic sur Enregistrer, `_enregistrer_piece()` :
 - vérifie le centre (comptable) ;
 - transforme chaque nom de tiers en code réel avec `dl.resoudre_tiers()` (création si nouveau) ;
 - reconstruit l'opération avec les codes définitifs ;
 - appelle `dl.enregistrer_operation()` (ou `dl.remplacer_piece()` pour une correction).
5. `logic/donnees.py` contrôle la date, numérote, insère les lignes dans une transaction ; la base vérifie l'équilibre au commit.

Les valeurs saisies sont gardées en JSON dans `ecritures.valeurs_json` : c'est ce qui permet de rouvrir une pièce dans le formulaire pour la corriger, et au tableau de bord de lire le motif d'un transfert.

5. Architecture du code

5.1 Les technologies

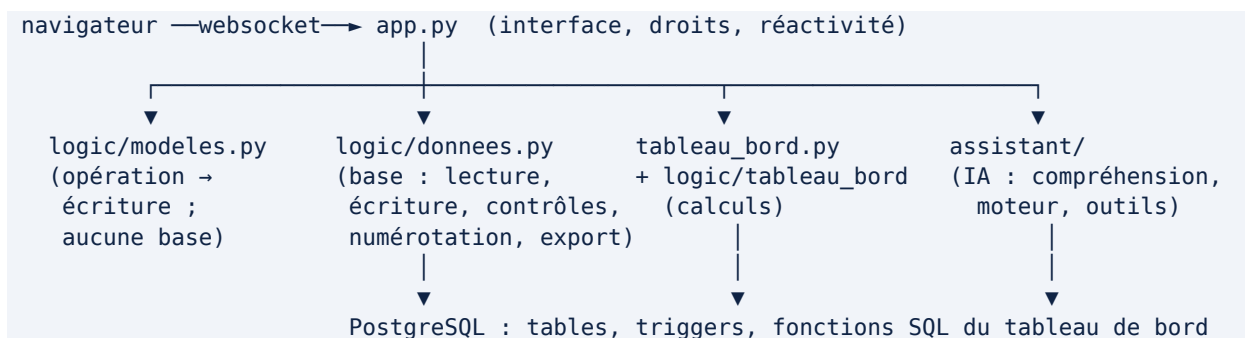
Élément	Choix	Version figée
Langage	Python	3.14 (image Docker python:3.14-slim, poste d'Afiya 3.14.3)
Interface et serveur web	Shiny for Python	shiny==1.7.0, shinychat==0.6.1
Base de données	PostgreSQL	16 (image postgres:16)
Accès base	psycopg2 + pandas	psycopg2-binary==2.9.12, pandas==3.0.5
Assistant IA	chatlas + API Anthropic	chatlas[anthropic]==0.23.0
Graphiques de l'assistant	matplotlib	>=3.8, <4
Fautes de frappe (assistant)	rapidfuzz	>=3.0, <4
Codes d'accès	bcrypt	>=4.0
Configuration	python-dotenv	.env
Tests	pytest, httpx	
Déploiement	Docker Compose sur Linux	

Les versions sont **figées volontairement** dans `requirements.txt` : une dépendance qui change seule ne doit pas casser un déploiement. On les met à jour exprès, jamais par un `pip install --upgrade` en passant.

Shiny for Python est le portage Python de Shiny (R). Il faut connaître ses trois idées : les **inputs** (valeurs envoyées par le navigateur, `input.m_modele()`), les **calculs réactifs** (`@reactive.calc`, recalculés quand ce qu'ils lisent change) et les **sorties** (`@render.ui`, `@render.data_frame`, redessinées automatiquement). Le navigateur et le serveur restent connectés par un websocket.

Historique utile : l'application a d'abord été écrite en R (`R/modèles.R`) avec des classeurs Excel comme stockage, puis portée en Python sur PostgreSQL. Certains commentaires parlent encore de « la version Excel » ou « la version R ».

5.2 Les couches



- **logic/modeles.py** ne connaît pas la base. Il transforme des valeurs de formulaire en lignes d'écriture (DataFrame pandas). On peut le tester sans PostgreSQL.

- **logic/donnees.py** est la seule porte d'entrée vers la base pour l'application : pool de connexions, lecture du référentiel et des écritures, enregistrement, validation, renvoi, suppression, contrôles, soldes, export Sage, authentification.
- **app.py** construit l'interface, applique les droits et branche le tout. Il ne contient pas de SQL.
- **tableau_bord.py** (interface) et **logic/tableau_bord.py** (calculs) forment un module Shiny isolé, branché dans app.py par deux lignes.
- **assistant/** est un paquet indépendant, utilisé par l'onglet Assistant IA.
- **composants.py** : petites fonctions qui fabriquent des morceaux d'interface (titre de page, carte, filtre, indicateur).
- **www/** : CSS, polices, icônes, logos servis tels quels.

5.3 Démarrage

Au lancement de app.py :

1. `load_dotenv()` lit le `.env` (avant tout import de `logic.donnees`, qui crée le pool de connexions à l'import) ;
2. la journalisation est configurée (`hakili.log` + sortie standard) ;
3. **logic/migrations.py joue les migrations SQL manquantes**. Si l'une échoue, l'application **refuse de démarrer** et nomme le fichier fautif. C'est voulu : une panne au déploiement vaut mieux qu'une panne découverte par une caissière ;
4. l'interface (`app_ui`) et le serveur (`server`) sont créés : `app = App(app_ui, server, static_assets=www/)`.

5.4 Une session utilisateur

`server(input, output, session)` est appelée une fois par onglet de navigateur ouvert. Elle contient tout l'état de cette session :

- `util` : l'utilisateur connecté (ou `None`) ;
- `ref()` : le référentiel (comptes, tiers, journaux, centres, utilisateurs, soldes d'ouverture) ;
- `donnees()` : les écritures de la **fenêtre de travail** (depuis le 1er janvier de l'année + toute pièce non exportée) ;
- `tresorerie()` : les mouvements cumulés par compte de trésorerie et par centre, pour les soldes (quelques dizaines de lignes, quel que soit l'historique) ;
- `correction` : la pièce en cours de correction ;
- l'objet de conversation de l'assistant.

`page()` affiche soit l'écran de connexion, soit la coquille de l'application. Les onglets sont construits **une seule fois par connexion** dans `corps()`, en lisant le référentiel sous `reactive.isolate()`. Sans cette précaution (corrigée le 10/09/2026), chaque pièce enregistrée par n'importe quel centre reconstruisait tous les onglets et effaçait le formulaire en cours de saisie.

5.5 Plusieurs postes en même temps : la table `revision`

Les cinq centres et le comptable travaillent en même temps. Pour qu'un écran voie ce qu'un autre poste vient d'enregistrer, PostgreSQL tient un compteur :

- la table `revision` a une seule ligne avec deux colonnes : `valeur` (écritures) et `referentiel` ;

- des triggers les incrémentent à chaque modification des tables concernées ;
- chaque session interroge cette ligne **toutes les 2 secondes** (`reactive.poll()`) ;
- tant que les valeurs ne bougent pas, rien n'est relu ; quand valeur change, `donnees()` est relu ; quand `referentiel` change, `ref()` est relu.

Une coupure de la base pendant ce sondage est absorbée : la dernière valeur connue est gardée et un avertissement va au journal (sinon Shiny fermait la session et l'écran devenait gris).

Le même sondage vérifie que le compte connecté est toujours actif.

5.6 Les règles qui protègent la réactivité

Ces règles ont chacune coûté un bug. Gardez-les :

- **Lire le référentiel sous `reactive.isolate()`** dans tout ce qui construit un formulaire (`m_champs`, `valeurs`, `operation`, `m_apercu`, `m_ruban`, `_ligne_repartition_ui`), sinon une écriture d'un autre centre reconstruit le formulaire et efface ce que la caissière tape.
- **Montants avec `update_on="blur"`** : la valeur n'est envoyée qu'en quittant le champ.
- **Identifiants de champs Shiny** : lettres, chiffres et `_` uniquement. Un code journal avec un tiret (BDU-BF) passe par `_id_journal()`.
- **Effets créés dynamiquement** (lignes de répartition) : détruire ceux d'une ligne retirée (`rep_effets_crees`), sinon ils s'accumulent.
- **Tout contrôle d'accès se refait dans l'effet serveur** (`_valider`, `_rejeter`, `_supprimer`, `_creer_lot...`), jamais seulement en cachant un bouton.
- **Élargir aux pièces liées avant le contrôle de centre** (`dl.avec_liees()` puis `_hors_centre()`), sinon on peut agir sur la moitié d'un autre centre.

5.7 Accès à la base

- Un `ThreadedConnectionPool` (1 à 20 connexions) créé à l'import de `logic/donnees.py`, avec `DATABASE_URL`.
- Tout passe par le gestionnaire de contexte `_connexion()` : `commit` si le bloc réussit, `rollback` sinon, et une connexion morte est retirée du pool au lieu d'y être rendue.
- Les opérations qui touchent plusieurs lignes (validation, correction, export) se font **dans une seule transaction**, avec `SELECT ... FOR UPDATE` sur les lignes concernées pour qu'un autre utilisateur attende au lieu d'écrire en même temps.
- Les compteurs de numérotation sont des `INSERT ... ON CONFLICT DO UPDATE ... RETURNING :` atomiques sans verrou applicatif.
- Les requêtes sont toujours paramétrées (`%s`), jamais construites par concaténation de valeurs saisies.

5.8 Un seul processus

En production, Shiny tourne dans **un seul processus** (`shiny run --host 0.0.0.0 --port 8022 app.py`). Un calcul long bloque tout le monde pendant sa durée. C'est pour ça que les contrôles ont été vectorisés (52 s → 0,7 s sur 25 000 pièces) et que le tableau de bord calcule le flux une seule fois par appel (35 s → 5 s). Si l'application ralentit un jour, c'est le premier endroit à regarder (voir chantiers : `reactive.extended_task`, plusieurs workers avec sessions collantes côté nginx).

5.9 Journalisation

logging écrit dans `hakili.log` (ou `HAKILI_LOG_FILE`) et sur la sortie standard. Loggers : `hakili.app`, `hakili.donnees`, `hakili.modeles`, `hakili.migrations`, `hakili.assistant`, `hakili.tableau_bord`. Sont tracés : connexions et échecs, verrouillages, pièces enregistrées, corrigées, supprimées, validées, renvoyées, lots d'export, réimports, créations de comptes et de tiers, clôtures.

En production, lisez-les avec `docker compose logs app`. Le fichier `hakili.log` dans le conteneur est perdu à chaque reconstruction (pas de rotation) : voir chantiers.

5.10 Le serveur MCP (facultatif)

`mcp_server/server.py` expose les outils de l'assistant au protocole MCP, pour interroger la base depuis un autre client (Claude Desktop par exemple). **L'application ne l'utilise plus** depuis le 24/09/2026 : l'assistant appelle ses outils directement. Il peut être ignoré, ou supprimé si personne ne s'en sert.

6. La base de données PostgreSQL

Base de production : **Hakili_compta**, utilisateur **hakili_admin**, dans le conteneur db (PostgreSQL 16). Les données vivent dans le volume Docker externe **hakili_gestion_caisse_pgdata**.

Le principe de conception : tout ce que la base peut garantir, elle le garantit elle-même (clés, contraintes, triggers), pour qu'un script ou une erreur de code ne puisse pas casser la comptabilité. Les contrôles Python ne servent qu'à donner un message lisible avant.

6.1 Les tables métier

comptes — plan comptable

compte (clé, 6 chiffres), intitulé, nature (charge, produit, trésorerie, bilan, tiers), tiers_obligatoire (oui/non), dépense_courante (oui/non : proposé dans « Dépense courante »), nb_2024_2025 (fréquence historique, informative).

tiers — plan tiers

code_tiers (clé, 17 caractères au plus), intitulé, compte_collectif (→ comptes), type (client, fournisseur, personnel), actif, actif_annee (présent dans les listes de l'année en cours).

journaux

journal (clé : CP, CMD, Banque, BDU-BF, VTE, ACH), intitulé, compte_contrepartie (compte de caisse ou de banque, vide pour VTE/ACH), type (trésorerie ou opérations), prefixe_piece (début des numéros définitifs : CP, CMD, CBI...), solde_ouverture (banque seulement), caisse_physique (oui pour CP et CMD), actif.

centres

code_centre (clé : SIA, TAM, SAA, PIS, NAG, SIE), intitulé, section_analytique (code Sage sur 4 caractères), ordre (ordre de numérotation), actif.

soldes_ouverture_centre

Un solde d'ouverture par couple (centre, journal) pour CP et CMD. Saisi depuis Référentiel > Paramètres.

utilisateurs

identifiant (clé), nom, rôle (saisie ou validation), centre, code_acces (hash bcrypt), tentatives_echouees, verrouille_jusqu_a, doit_changer_code, actif. Le comptable = rôle validation sur le centre SIE.

ecritures — la table centrale

Une ligne d'écriture par enregistrement.

Colonne	Rôle
id_ligne	clé, {id_piece}-{n}
id_piece	identifiant de la pièce : {centre}-{horodatage}-{AAMM}-{nnn}

Colonne	Rôle
id_lien	pièces solidaires (approvisionnement, versement) : L{horodatage} - {8 car. aléatoires}
num_provisoire	{centre} - {AAMM} - {nnn}
num_definitif	{préfixe}{AAMM}{nnn}, donné à la validation
num_reserve	numéro gardé par une pièce renvoyée, rendu à la revalidation
journal, centre, date_piece, compte, code_tiers, libelle	
debit, credit	numeric(14,2), positifs, jamais les deux sur une ligne
modele	id du modèle de saisie
saisi_par, saisi_le, valide_par, valide_le, exporte_le	traçabilité
statut	saisie, a_corriger, validee, exportee
observation	note libre ou motif de renvoi
valeurs_json	valeurs du formulaire (pour corriger la pièce, lire le motif d'un transfert)
reference_transfert, centre_contrepartie	transferts entre centres
id_lot	lot d'export qui contient la pièce

Tables de service

Table	Rôle
compteurs	numérotation atomique : clés prov : {centre} : {AAAAMM}, def : {journal} : {AAAAMM}, trf : {AAAAMM}
suppressions_écritures	contenu complet (JSON) de toute pièce supprimée ou remplacée, avec qui et quand
revision	une ligne, deux compteurs (valeur, referentiel) sondés toutes les 2 s
libelles_types	libellés suggérés par compte
lots_export	chaque lot d'export : auteur, date, nombre de pièces et de lignes, total, période, fichier exact (bytea)
reimports_sage	chaque réexport volontaire : période, motif, fichier
periodes_cloturees	mois clôturés (AAAA-MM)
schema_migrations	migrations déjà jouées

Tables du tableau de bord

Table	Rôle
rubriques_tableau	les lignes du compte de résultat et leur bloc (encaissement, charge, hors_exploitation, exclu)
classement_comptes	règles qui rattachent un compte (par préfixe), éventuellement un mot du libellé ou un modèle, à une rubrique

Table	Rôle
parametres_centre	cible de marge (15 % par défaut) et de trésorerie (2 mois de charges) par centre

Tables de l'assistant IA

Table	Rôle
centres_alias	façons d'écrire un centre (« saab », « tampuy »)
categories_chargée	catégories de dépense, comptes et mots-clés associés
journal_assistant	chaque question : utilisateur, outils appelés, réponse, durée, jetons

6.2 Les garde-fous en base

Objet	Ce qu'il garantit
Clés étrangères	pas de compte, journal, centre ou collectif inconnu
ck_montants_positifs	débit et crédit ≥ 0 , jamais les deux sur une ligne
trg_equilibre_piece (trigger différé)	chaque pièce est équilibrée au moment du commit
trg_numero_unique (trigger différé)	deux pièces d'un journal n'ont jamais le même numéro définitif
uq_transfert_entree (index unique partiel)	une sortie de transfert n'est rapprochée que d'une seule entrée
trg_proteger_ecritures	une ligne exportée ne se modifie ni ne se supprime ; un mois clôturé ne reçoit rien et ne change plus
ck_tiers_longueur_sage	code tiers ≤ 17 caractères (posée seulement si aucun code existant ne dépasse)
Rôle hakili_lecture	rôle sans connexion, qui ne lit que quelques tables ; la requête libre de l'assistant s'exécute sous ce rôle

Les triggers d'équilibre et de numéro sont **différés** (DEFERRABLE INITIALLY DEFERRED) : une pièce n'est équilibrée qu'après l'insertion de toutes ses lignes.

Échappatoire volontaire pour un script de reprise exceptionnel, jamais utilisée par l'application :

```
BEGIN;
SET LOCAL hakili.autoriser_modif_protegee = 'oui';
-- modification exceptionnelle, réfléchie, sauvegardée avant
COMMIT;
```

6.3 Les fonctions SQL

Fonction	Utilisée par	Rôle
verifier_equilibre_piece(), verifier_numero_unique(), proteger_ecritures()	triggers	voir ci-dessus

Fonction	Utilisée par	Rôle
bump_revision(), bump_revision_referentiel(), bump_revision_utilisateurs()	triggers	incrémenter revision
tb_normaliser(text)	tableau de bord	majuscules sans accents
flux_tableau_bord(debut, fin, centres[])	tableau de bord, assistant	chaque ligne qui fait face à un mouvement de caisse ou de banque, avec sa rubrique, son sens et son montant (positif = argent entré)
tb_rubrique_motif_transfert(motif)	flux_tableau_bord	rubrique d'un transfert selon son motif
soldes_tableau_bord(fin, centres[])	tableau de bord	solde des caisses par centre et de la banque à une date
soldes_fin_mois(debut, fin, centres[])	tableau de bord	solde des caisses en fin de chaque mois
Vue v_lignes	assistant	écritures enrichies (noms, nature du compte, année scolaire, est_transfert)

6.4 Les migrations

sql/schema.sql + sql/seed.sql créent une base **neuve** (ils sont joués automatiquement par l'image PostgreSQL uniquement quand le volume est vide). Toute évolution d'une base existante passe par un fichier dans **sql/migrations/**, nommé AAAA-MM-JJ_sujet.sql et joué dans l'ordre alphabétique.

logic/migrations.py les joue **au démarrage de l'application**, chacune dans sa transaction, et inscrit son nom dans schema_migrations. Une migration qui échoue empêche le démarrage.

Règles :

1. **Une migration déjà jouée ne se modifie jamais.** Pour corriger, on écrit une nouvelle migration (exemple : 2026-10-07b_motif_transfert.sql à côté de 2026-10-07_camp_de_vacances.sql).
2. Toute migration est **idempotente** : IF NOT EXISTS, ON CONFLICT DO NOTHING, CREATE OR REPLACE, contraintes testées dans pg_constraint.
3. Une migration ne supprime ni ne modifie d'écritures comptables.
4. Mettez aussi à jour schema.sql / seed.sql si une base neuve doit naître avec l'objet, et vérifiez que les deux chemins (base neuve, base migrée) donnent le même résultat.
5. Avant de déployer une migration qui pose une contrainte, vérifiez que les données de production la respectent (requête de contrôle en tête de fichier).

Fichier	Contenu
2026-09-05_durcissement_auth.sql	verrouillage après échecs, table des suppressions
2026-09-10_suppression_code_acces_centres.sql	retrait d'un ancien code de centre en clair
2026-09-11_soldes_par_centre_et_banque_cbi.sql	soldes d'ouverture par centre, journal Banque (CBI, 521200), BDU-BF inactif

Fichier	Contenu
2026-09-12_transferts_internes.sql	colonnes de transfert entre centres
2026-09-15_rattrapage_production.sql	rattrapage des quatre migrations oubliées en production
2026-09-22_correctifs_audit.sql	contrainte de montants, trigger d'équilibre, index des transferts
2026-09-23_sections_sage.sql	sections analytiques sur 4 caractères
2026-09-24_assistant_ia.sql	alias de centres, catégories, journal de l'assistant, vue v_lignes
2026-09-25_assistant_libelles.sql	libellés des catégories dans les mots du directeur
2026-09-25_numerotation_date_centre.sql	ordre des centres, numéro réservé, unicité des numéros
2026-10-02_tableau_bord.sql	rubriques, classement, flux_tableau_bord, soldes_tableau_bord
2026-10-03_tableau_bord_direction.sql	cibles par centre, flux enrichi, soldes_fin_mois
2026-10-03b_tableau_bord_equipement_camp.sql	règles équipement et camp
2026-10-07_camp_de_vacances.sql	règle « CAMP » en mot entier
2026-10-07b_motif_transfert.sql	motif des transferts lu avant le libellé
2026-10-08_correctifs_audit.sql	lots d'export, clôtures, protection des pièces exportées, révisions séparées, 17 caractères, rôle de lecture, code à changer
2026-10-08b_reimports_sage.sql	table des réimports

Les deux dernières **ne sont pas encore jouées en production** au 08/10/2026 (code non déployé).

6.5 Les scripts SQL hors migrations

Fichier	Usage	Danger
sql/schema.sql, sql/seed.sql	base neuve	ne jamais rejouer en production
sql/ charger_plan_comptable.sql , sql/charger_plan_tiers.sql	chargement des plans Sage (fait le 18/09/2026)	contiennent des noms réels, hors Git , copiés par SCP puis effacés du serveur
sql/seed_tiers.sql	ancien jeu de tiers	hors Git
sql/ diagnostic_production.sql	lecture seule : objets manquants	sans danger
sql/production_reset.sql, sql/purge_avant_mise_en_service.sql	vident les écritures (utilisés une fois avant la mise en service)	détruisent la comptabilité réelle , ne jamais lancer

6.6 Se connecter à la base

En production (depuis le dossier ~/hakili_gestion_caisse, psql n'est pas installé sur l'hôte) :

```
docker compose exec db psql -U hakili_admin -d Hakili_compta
```

Requêtes utiles :

```
-- état général
SELECT statut, count(DISTINCT id_piece) FROM ecritures GROUP BY statut;
-- pièces déséquilibrées (doit être vide)
SELECT id_piece FROM ecritures GROUP BY id_piece HAVING round(sum(debit),2) <>
round(sum(credit),2);
-- migrations jouées
SELECT * FROM schema_migrations ORDER BY version;
-- codes tiers trop longs
SELECT code_tiers FROM tiers WHERE length(code_tiers) > 17;
-- pièces encore rattachées au Siège
SELECT DISTINCT id_piece, date_piece FROM ecritures WHERE centre = 'SIE';
```

En local, utilisez pgAdmin ou psql avec le DATABASE_URL de votre .env.

7. Carte des fichiers du projet

Dossier de travail sur le PC d'Afiya : C:\Users\USER\Desktop\STAGE_COMPTABILITE\

HAKILI_LAB_postgres_2. Dépôt Git : <https://github.com/imaneodg/hakili-lab-comptabilite>. Sur le serveur : ~/hakili_gestion_caisse.

Légende : **Git** = versionné ; **local** = ignoré par Git, ne quitte pas le poste.

7.1 Racine

Fichier	Git	Rôle
app.py	Git	L'application Shiny : interface, connexion, droits, tous les onglets sauf le tableau de bord. ~3 750 lignes.
composants.py	Git	Petites briques d'interface (titre_page, carte_bandeau, filtre, stat, hk_info, format_montant). Aucune logique.
tableau_bord.py	Git	Interface et serveur du module Tableau de bord (module Shiny tb).
requirements.txt	Git	Dépendances Python, versions figées.
Dockerfile	Git	Image de l'application (Python 3.14 slim, port 8022).
docker-compose.yml	Git	Production réelle : services db (postgres:16) et app, volume externe.
.env.example	Git	Modèle de configuration, sans aucune vraie valeur.
.env	local	Vraie configuration (base, clé Anthropic). Ne jamais versionner.
.gitignore, .dockerignore	Git	Exclusions. Très commenté : lisez-le, il explique pourquoi les classeurs Excel et certains scripts SQL sont exclus (noms réels d'élèves).
conftest.py	Git	Configuration pytest : charge .env, fixe la date du jour des tests au 01/01/2100 (les pièces de test sont datées 2098-2099).
README.md	Git	Ancienne présentation. En partie périmée (parle du port 8000 et d'un compose sans base). Ce dossier docs/ la remplace.
import_historique.py	Git	Importe les brouillards Excel 2026 dans une base. Servait à construire la base de démonstration locale. Pas pour la production (décision du 16/09/2026).
verifier.py	Git	Recrée une base de test jetable hakili_test depuis schema.sql + seed.sql, puis lance toute la suite pytest. Ne touche jamais Hakili_compta.
verifier_donnees.py	Git	Importe les brouillards dans une base hakili_verif et contrôle la chaîne d'analyse et les graphiques.
nettoyer_operations.py	Git	Vieux script qui vide les écritures d'une base. Dangereux , à supprimer ou à ne jamais lancer.
demo_manuel.py, mock_ocr_server.py	Git	Vides : restes d'un essai d'intégration OCR abandonné. À supprimer.
logo_Hakili_Lab.jpeg	Git	Logo d'origine.
HAKILI_LAB_postgres_2.code-workspace	local	Espace de travail VS Code d'Afiya.
hakili.log	local	Journal de l'application en local.
Plan_comptable.xlsx, Plan_tiers.xlsx	local	Exports Sage du plan comptable (1 039 comptes) et du plan tiers (1 454 tiers). Noms réels : jamais sur GitHub.

Fichier	Git	Rôle
BROUILLARD_*_2026.xlsx	local	Brouillards de caisse 2026 de Saaba et Tampouy (CP et CMD). Données personnelles.
.venv/, __pycache__/, .pytest_cache/	local	Environnement Python et caches.

7.2 logic/ — le métier

Fichier	Rôle
modeles.py	Les modèles de saisie et leur traduction en écritures. Pas d'accès à la base. Voir chapitre 4.
donnees.py	Toute la couche base : pool de connexions, référentiel, écritures, enregistrement et correction, tiers, authentification et codes, numérotation, validation, renvoi, suppression, soldes, contrôles, export Sage (lots, réimports), clôtures, révisions. ~2 000 lignes.
migrations.py	Joue sql/migrations/*.sql au démarrage.
tableau_bord.py	Calculs du tableau de bord (fonction unique calculer()).
analyse.py	Anciennes fonctions d'analyse financière déterministes (recettes, dépenses, masse salariale...). Encore utilisées par l'assistant (assistant/semantique.py en reprend les règles de transfert et de charges).

7.3 assistant/ — l'assistant IA

Fichier	Rôle
__init__.py	Description des couches.
config.py	Réglages lus dans l'environnement (modèle, longueur des réponses, historique, bornes SQL).
comprehension.py	Texte libre → centre, période, tiers, catégorie (fautes tolérées : « saab », « fevirer »).
semantique.py	Écritures → faits (encaissements, décaissements, charges) avec une seule définition par chiffre.
indicateurs.py	Les indicateurs disponibles, dont ceux repris du tableau de bord.
moteur.py	Les calculs : analyser, comparer, soldes, lister, contrôles, personnel, point financier, requête SQL en lecture.
outils.py	Les 12 outils exposés au modèle d'IA, avec la portée de l'utilisateur.
rendu.py	Tableaux et graphiques (matplotlib) affichés dans les cartes du chat.
prompt.py	Consignes données au modèle, reconstruites à chaque question.
session.py	Une conversation par utilisateur : client Anthropic, historique borné, verrou, journal, messages d'erreur lisibles.
questions.py	Questions suggérées.
referentiel.py	Lecture du référentiel pour l'assistant.
texte.py	Dates et mois en français.

Fichier	Rôle
journal.py	Écriture dans journal_assistant.

7.4 sql/

Voir chapitre 6 : schema.sql, seed.sql, migrations/, diagnostic_production.sql, et les scripts dangereux production_reset.sql et purge_avant_mise_en_service.sql. Les scripts de chargement des plans (charger_plan_*.sql, seed_tiers.sql) sont locaux.

7.5 outils/ — scripts d'administration

Fichier	Usage
charger_plans.py	Exécute les scripts de chargement des plans sans psql (Windows).
comparer_plans.py	Lecture seule : compare la base aux classeurs Sage.
verifier_referentiel.py	Lecture seule : ce que contient vraiment le référentiel.
corriger_camp_2026.py	Remet la marque du camp sur les pièces importées des brouillards (base de démo). Simulation par défaut, - -appliquer pour écrire.
definir_code.py	Pose le code d'un utilisateur depuis le serveur, sans connaître l'ancien. Seul moyen de reprendre la main si plus personne ne connaît le code du comptable : docker compose exec app python outils/definir_code.py comptable.
evaluer_assistant.py	Pose une série de vraies questions à l'assistant (vrai modèle, vraie base) et affiche outils, réponses et durées. Coûte quelques centimes.
tiers_trop_longs.sql	Liste les codes tiers > 17 caractères et explique comment les renommer.

7.6 backup/

Fichier	Git	Rôle
sauvegarde.sh	à commiter	La sauvegarde à utiliser : pg_dump via docker compose exec db, contrôle du fichier, conservation 30 jours, copie rclone optionnelle. À mettre dans le crontab du compte imane.
restauration_essai.sh	à commiter	Restaure le dernier dump dans une base temporaire, compare les comptes, supprime la base temporaire.
pg_backup.py, cron_sauvegarde_host.sh	Git	Ancienne méthode, inutilisable sur ce serveur (exige pg_dump sur l'hôte et sudo). Gardés pour mémoire, à supprimer à terme.
donnees_avant_correctifs_age_2026-09-23.py	Git	Copie de code versionnée par erreur. À supprimer.
app_avant_*.py, avant_*/, ancien_tableau_bord_*/	local	Copies de sécurité faites avant de grosses modifications. Inutiles maintenant que tout est dans Git.
sauvegardes/	local	Dumps de base. Jamais dans Git.

7.7 mcp_server/, services/, nginx/, www/, tests/

Chemin	Rôle
mcp_server/server.py	Façade MCP facultative sur les outils de l'assistant (non utilisée par l'application).
services/ocr_client.py, services/__init__.py	Vides. Projet d'OCR des factures abandonné (voir chapitre 13).
nginx/hakili.conf	Modèle de reverse proxy HTTPS. Pas utilisé : le nginx de production est celui de l'hôte, géré par l'administrateur. Utile comme base à lui transmettre.
www/app.css	Toute la charte : variables - - hk - *, barre latérale marine, cartes blanches, boutons.
www/tableau_bord.css	Styles du tableau de bord.
www/fonts/, www/bootstrap- icons/	Polices (Inter, Public Sans) et icônes, servies localement pour fonctionner sans Internet.
www/*.png, favicon.ico	Logos et icônes.
tests/	~300 tests pytest (voir chapitre 12). test_ocr_client.py est vide.

7.8 Claude outputs/ — à manipuler avec précaution

Dossier **local** où ont été déposés les rapports d'audit, des captures d'écran, des fichiers de test Sage et des études de design. Il contient aussi un fichier **env avec le mot de passe PostgreSQL de production en clair**.

- Ne le copiez pas, ne l'envoyez pas, ne le versionnez jamais.
- Les documents utiles qu'il contient sont repris dans ce dossier docs/ et dans le projet claud.ai « tu ».
- Recommandation : supprimer ce fichier env une fois la passation faite, puis changer le mot de passe de la base (chapitre 11).

7.9 Hors du dossier

- Le **projet claud.ai « tu »** contient l'historique détaillé du travail : audits (18/09, 22/09, 25/09, 07/10), comptes rendus de correctifs, de déploiements, de l'import Sage, du tableau de bord et de l'assistant. Ce dossier en est la synthèse ; les originaux restent utiles pour le détail d'une décision.
- Le **dossier Sage 100 « HAKILI LAB SARL COMPTABILITE DEFINITIF 2026 »** et son format d'import HAKILI_ECRITURES sont sur le poste du comptable (emplacement à préciser par Afiya).

8. Le tableau de bord

Fichiers : `tableau_bord.py` (interface), `logic/tableau_bord.py` (calculs), `www/tableau_bord.css`, migrations `2026-10-02_tableau_bord.sql`, `2026-10-03_tableau_bord_direction.sql`, `2026-10-03b_`, `2026-10-07_`, `2026-10-07b_`. Tests : `tests/test_tableau_bord.py`, `tests/test_camp_cibles_forces.py`.

État : sur la branche `tableau-bord-v2` (poussée sur GitHub), pas encore fusionnée dans `main` ni déployée au 08/10/2026.

8.1 À quoi il sert

Le directeur voulait, pour chaque centre, l'analyse de « santé financière » qu'on avait faite à la main sur le brouillard de Tampouy : combien le centre reçoit, combien il dépense, s'il gagne de l'argent, s'il paie trop de salaires, combien d'élèves paient, combien il reste en caisse, et si les chiffres sont fiables. Le tableau de bord refait ces calculs pour **tous les centres, avec les mêmes règles**, directement sur les écritures saisies.

Il est **réservé au comptable** pour l'instant (vérifié côté serveur par `req(autorise())`). Une version pour le directeur est prévue.

8.2 Ce qu'on voit

En haut, toujours visibles : la **période** (du... au..., par défaut du 1er janvier à la fin du dernier mois avec des opérations) et les **centres** (un ou plusieurs ; le Siègne n'en fait jamais partie).

Vue d'ensemble - un bandeau par centre avec **trois chiffres** : Résultat, Sorties hors exploitation, Argent disponible (caisses ; la banque est affichée à part, elle est commune) ; - les **points à regarder** : alertes automatiques classées par gravité ; - un **comparatif des centres** avec des pastilles (Correct, À surveiller, Attention, Repère — toujours un mot, jamais la couleur seule) ; - le **compte de résultat mensuel en encaissements** (replié), avec le détail par rubrique.

Fiche centre, quatre onglets : - **Résultat** : compte de résultat mois par mois, salaires comparés au plafond salarial ; - **Élèves** : élèves payants par mois, frais moyen, seuil de rentabilité en élèves, retards de paiement ; - **Caisse** : solde fin de mois, mois de charges couverts, capital de base, écart entre caisse attendue et caisse constatée ; - **Fiabilité** : pièces validées, semaines sans écriture, encaissements sans élève, dates impossibles, compte d'attente, comptes non classés, pièces non validées — avec la liste des opérations concernées.

Les **cibles** de chaque centre (marge visée, capital de base en mois de charges) se règlent depuis l'écran et sont gardées dans `parametres_centre`. Bornes : 0 à 50 % de marge, 0 à 12 mois.

8.3 Les règles de calcul

Base caisse : on compte l'argent à la date où il entre ou sort d'une caisse ou de la banque. Une seule source : la fonction `SQL flux_tableau_bord(début, fin, centres)`.

Pour chaque pièce qui touche une caisse ou la banque, la fonction renvoie chaque ligne **en face** de la trésorerie (le compte de contrepartie), avec : - son montant signé (positif = argent entré) ; - sa **rubrique**, trouvée dans `classement_comptes`.

Le classement (classement_comptes) rattache un compte à une rubrique par **préfixe** (6, 62, 622200...), éventuellement conditionné par un **mot du libellé** (motifs) ou un **modèle de saisie**. Ordre de priorité quand plusieurs règles correspondent : 1. une règle avec motif passe avant une règle sans motif ; 2. une règle liée à un modèle passe avant une règle générale ; 3. le préfixe le plus long gagne ; 4. puis le champ ordre.

Exceptions : - un **transfert entre centres** saisi depuis le 07/10/2026 est classé par son **motif** (valeurs_j son - >> 'motif'), avant toute règle de libellé ; - le **camp de vacances** est reconnu **uniquement au libellé** (CAMP suivi d'un espace, FRAIS CV, AVANCE CV, SOLDE CV), jamais au mois : un cours donné en juillet reste un cours ; - les virements CP → CMD et caisse → banque sont exclus (rubrique transfert).

Les rubriques (rubriques_tableau) et leur bloc :

Bloc	Rubriques
Encaissements d'exploitation	Cours d'appui, Camp de vacances, Frais de document, Contributions reçues des centres (SIAO)
Charges d'exploitation	Vacations et salaires, Contribution au SIAO, Loyer, Produits d'entretien, Électricité et eau, Internet et communication, Fournitures et impressions, Autres charges courantes, Frais financiers
Hors exploitation	Impôts et taxes, Remboursements d'emprunts / Emprunts reçus, Prêts entre centres, Dossiers de conseil, Achats d'équipement, Avances au personnel
Contrôle	Sorties/encaissements en attente de classement (471), non classés
Exclu	Transferts entre caisses

Un centre est toujours vu seul : sa contribution au SIAO est une charge, quels que soient les autres centres choisis. **L'ensemble**, lui, ne compte pas deux fois un flux entre deux centres sélectionnés (colonne centre_contrepartie).

Les indicateurs

Indicateur	Formule
Résultat	encaissements d'exploitation – charges d'exploitation
Marge	résultat ÷ encaissements
Plafond salarial	encaissements – (charges – salaires) – marge cible × encaissements : ce qu'on peut payer en salaires en gardant la marge visée
Capital de base	cible en mois × charges d'exploitation moyennes par mois (2 mois par défaut)
Reste à constituer / disponible pour distribuer	écart entre l'argent en caisse et le capital de base
Élèves payants « typiques »	médiane des mois scolaires (hors mois où le camp fait plus de la moitié des recettes)
Seuil de rentabilité en élèves	charges moyennes mensuelles ÷ frais moyen par élève
Mois du cours	lu dans le libellé (assistant.semantique.mois_du_libelle), un paiement multi-mois réparti à parts égales
Caisse attendue	caisse de début + résultat – hors exploitation + transferts – versements en banque ; l'écart avec la caisse constatée révèle une pièce manquante

Toutes les pièces comptent, y compris celles pas encore validées ; leur nombre est affiché dans Fiabilité.

8.4 Modifier le classement

Le classement est **dans la base**, pas dans le code : pour qu'un nouveau mot-clé ou un nouveau compte tombe dans la bonne rubrique, on ajoute une ligne à `classement_comptes` par une **nouvelle migration** (jamais à la main en production sans trace). Un éditeur dans l'interface a été écarté volontairement : trop facile de casser le classement par erreur.

Exemple :

```
INSERT INTO classement_comptes (prefixe, motifs, modele, rubrique, ordre)
VALUES ('401', '{GARDIENNAGE}', '', 'vacations', 15)
ON CONFLICT (prefixe, motifs, modele) DO NOTHING;
```

8.5 Les limites connues

- **Mots-clés en sous-chaîne** : CANAL reconnaît « CANALISATION », MEGA « OMEGA », PRET « INTERPRETE ». Seul CAMP est traité en mot entier. La solution générale est une recherche par mot entier (~ '\m...\M') dans `flux_tableau_bord`.
- **Règle à motif sur préfixe court** : 6 + CONSEIL passe avant 632710 vacations. Une vacation libellée « CONSEIL DE CLASSE » part en dossiers de conseil.
- **Règlement fournisseur** : la nature de la charge n'existe que dans le libellé libre ; le classement la devine. Un champ « Nature » dans le formulaire réglerait le problème à la source.
- **Impôts et taxes** en hors exploitation, alors qu'en SYSCOHADA ce sont des charges d'exploitation : décision reportée, une ligne SQL à jouer quand la direction aura tranché.
- La description du contrôle « Dates aberrantes » à l'écran (`tableau_bord.py`, liste `CONTROLES`) parle encore de « plus d'un an avant la période » alors que la règle est maintenant fixe (avant le 01/01/2025 ou dans le futur). Texte à corriger.
- Un calcul prend environ 5 secondes sur deux ans de données, pendant lesquelles le processus unique est occupé.
- À venir (demandé) : version directeur, prévision de trésorerie sur 3 mois.

8.6 Chiffres de référence (base de démonstration)

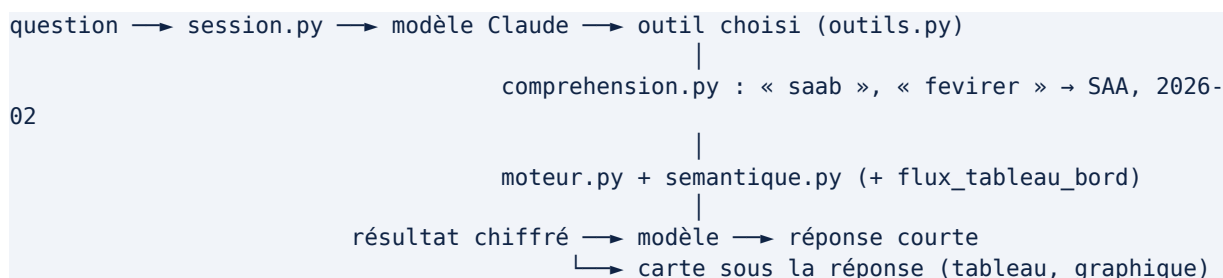
Sur la base locale construite à partir des brouillards 2026, janvier à août 2026 : Tampouy reçoit 6 509 450 F, résultat 714 567 F (11,0 %), qui passe à 839 567 F (12,9 %) une fois le prêt de juin au SIAO corrigé en prêt ; Saaba reçoit 4 613 590 F, résultat 191 605 F (4,2 %). Utile pour vérifier qu'une modification ne change pas les chiffres sans raison.

9. L'assistant IA

Dossier assistant/, onglet « Assistant IA » (réservé au comptable). Reconstitué entièrement le 24/09/2026, réorienté vers le directeur le 25/09.

9.1 Le principe : le modèle ne calcule rien

Les nombres viennent toujours de fonctions Python ou SQL déterministes. Le modèle de langage (Claude, par l'API Anthropic) fait trois choses seulement : comprendre la question, choisir l'outil et ses paramètres, puis formuler la réponse avec le résultat. C'est ce qui rend les réponses vérifiables : la même question donne le même chiffre que le tableau de bord ou qu'une requête SQL.



9.2 Les outils

Outil	Ce qu'il fait
contexte	date du jour, période couverte par les données, centres
point_financier	la situation en un coup d'œil (argent disponible, reçu et dépensé du mois, comparaison, point à vérifier)
soldes	caisses par centre et banque à une date
analyser	un ou plusieurs indicateurs sur une période, regroupés par centre, mois, type...
comparer	deux périodes, avec l'écart
lister_écritures	derniers mouvements
chercher	recherche approximative d'un élève, fournisseur, enseignant ou catégorie
controles	doublons, compte d'attente, transferts, etc.
a_verifier	tous les points à vérifier en une liste
personnel	vacations et salaires par personne, avances
graphique	graphique tracé à partir d'un résultat déjà calculé (jamais de chiffres fournis par le modèle)
requete_sql	dernier recours : une requête SELECT en lecture seule, sous le rôle hakili_lecture, limitée à 10 s et 300 lignes

Une valeur mal comprise (centre ambigu, période illisible) n'est pas une erreur : l'outil renvoie les candidats et le modèle pose la question.

Mêmes chiffres que le tableau de bord (décision du 08/10/2026) : pour « bénéfice, résultat, marge, charges », l'assistant utilise les indicateurs `resultat_tableau`, `charges_exploitation`, `encaissements_exploitation`, `marge_tableau_pct`, `hors_exploitation`, calculés par `flux_tableau_bord`.

Les indicateurs « rattachés au mois du cours » existent encore mais ne servent que si on les demande explicitement.

9.3 Les consignes au modèle

assistant/prompt.py, reconstruit à chaque question (la date change). Points principaux : répondre au directeur, qui n'est pas comptable ; le chiffre d'abord, en une ou deux phrases, avec un repère ; jamais de numéro de compte ni de code de centre ; vocabulaire simple (« argent reçu » plutôt qu'« encaissements », « bénéfice » plutôt que « résultat ») ; pas de formules de politesse ni d'emoji. Lisez-le avant de modifier quoi que ce soit : chaque phrase corrige un défaut observé.

9.4 Sécurité et coûts

- **Portée** : le comptable voit tous les centres. Le code prévoit une portée limitée à un centre pour un futur utilisateur non comptable.
- **Requête SQL libre** : exécutée sous le rôle PostgreSQL hakili_lecture (lecture de v_lignes, centres, comptes, tiers, journaux, categories_charge, centres_alias, soldes_ouverture_centre seulement), en transaction lecture seule, avec un filtre supplémentaire contre query_to_xml et consorts. Si le rôle n'a pas pu être créé, l'outil refuse de s'exécuter.
- **Historique** : 8 échanges gardés par conversation (HAKILI_IA_HISTORIQUE), pour un coût stable.
- **Réponse** : 2 000 jetons au plus (HAKILI_IA_MAX_TOKENS).
- **Traçabilité** : chaque question est enregistrée dans journal_assistant (utilisateur, question, outils, réponse, durée, jetons, erreur).

9.5 Configuration

Dans .env :

Variable	Obligatoire	Rôle
ANTHROPIC_API_KEY	oui	clé de l'API Anthropic (compte à préciser par Afiya, voir chapitre 0)
HAKILI_MODELE_IA	non	nom du modèle (défaut dans assistant/config.py). Si l'assistant répond « Le modèle d'IA configuré est introuvable », c'est ce nom qu'il faut mettre à jour avec un modèle actuel de la documentation Anthropic
HAKILI_IA_MAX_TOKENS, HAKILI_IA_HISTORIQUE	non	voir ci-dessus
HAKILI_ASSISTANT_DATABASE_URL	non	connexion dédiée en lecture seule ; sinon la connexion de l'application, en transaction READ ONLY

Les pannes de l'API (clé refusée, surcharge, réseau) sont traduites en phrases lisibles dans session.py:message_erreur.

9.6 Tester l'assistant

- Tests sans appel au modèle : tests/test_assistant_ia.py, tests/test_assistant_hakili.py.
- Avec le vrai modèle et la vraie base (coûte quelques centimes) : python -m outils.evaluer_assistant, ou avec une question précise : python -m outils.evaluer_assistant

"combien saab a encaisse en fevirer". À lancer après toute modification de `prompt.py` ou des outils.

9.7 Compléter sans toucher au code

- Une nouvelle façon d'écrire un centre : `INSERT INTO centres_alias (alias, code_centre) VALUES ('tampy', 'TAM');`
- Une nouvelle catégorie de dépense, ou un mot-clé : table `categories_charge`.

Faites-le par une migration, pour que la production et votre base locale restent identiques.

10. L'export vers Sage 100

Sage 100 Comptabilité i7 est le livre officiel. L'application lui envoie les pièces **validées** sous forme d'un fichier texte, importé par un format paramétrable. Ce chapitre réunit tout ce qui a été appris en septembre 2026, souvent à la dure.

10.1 Le fichier produit

Fonctions `format_sage()` et `texte_fichier_sage()` dans `logic/donnees.py`.

- Texte **Windows-1252** (cp1252), un caractère impossible devient ? au lieu de faire planter l'export ;
- fins de ligne **CRLF** (\r\n). En LF, Sage lit tout le fichier comme une seule ligne et n'importe rien, sans aucun message ;
- séparateur **point-virgule**, pas d'en-tête, pas de guillemets ;
- **9 colonnes, dans cet ordre** :

#	Colonne	Exemple	Remarque
1	Journal	CP	
2	Date	01042026	jjmmaaaa
3	Pièce	CP2604008	numéro définitif
4	Compte	411000	6 chiffres
5	Tiers	411FRANCELINE	vide si aucun (d'où des ; ;)
6	Libellé	FRAIS CA MAR - ...	;, " et retours à la ligne retirés
7	Débit	0,00	virgule décimale
8	Crédit	36000,00	
9	Section	PSSY	<code>centres.section_analytique</code>

Une pièce est toujours exportée **entière et équilibrée** (la ligne de caisse globale des encaissements multi-mois est gardée : sans elle, Sage refusait la pièce).

10.2 Le dossier Sage : réglages indispensables

Sage impose des **longueurs fixes** décidées à la création du dossier. Trop court, il complète par des zéros ; trop long, il tronque ; **dans les deux cas sans prévenir**. Et la plupart de ces réglages se verrouillent dès qu'un enregistrement existe. D'où l'ordre : créer, régler, puis charger.

Réglage	Valeur
Longueur des comptes généraux	6
Longueur des comptes tiers	17 (sinon « La longueur du compte est incorrecte »)
Longueur des sections analytiques	4 (PSSY, TAMP, SAAB, SIA0, NAGR, SIEG)
Analytique	« Ignorer la préventilation » ; plan renommé CENTRES
Natures de comptes (Client, Caisse, Banque, charges, produits)	« Autoriser la saisie analytique » cochée
Exercice	01/01/2026 – 31/12/2026, franc CFA

Réglage	Valeur
Journaux	CP, CMD, Banque (521200), BDU - BF (521100), VTE, ACH. Vérifier que Sage accepte des codes de 6 caractères ; sinon renommer Banque en CBI ou BQ côté application
Compte 471000	N'existe pas dans Sage , n'y envoyez jamais une pièce qui le porte

Format d'import HAKILI_ECRITURES (Fichier > Format import/export paramétrable) : - type « Écritures comptables », fichier « Délimité » ; - les 9 champs dans l'ordre : Code journal, **Date de pièce** (format **j jmmaaaa**, pas j jmmaa), **N° pièce**, N° compte général, N° compte tiers, Libellé écriture, **Montant débit**, **Montant crédit**, N° section 1 ; - origine Windows, enregistrement Retour-chariot, séparateur Point-virgule, identifiant de texte aucun, 2 décimales, séparateur décimal virgule, pas de séparateur de milliers, pas d'entête ; - « **Interpréter les séparateurs identiques comme unique** » : **DÉCOCHÉ**. Cochée par défaut, elle fusionne les ; ; et décale toute la ligne.

Gardez une copie du fichier de format hors du dossier d'installation de Sage.

10.3 Les erreurs déjà rencontrées

Message Sage	Cause	Correction
Rien, aucune écriture	fichier en fins de ligne LF	export en CRLF (fait)
Lignes décalées d'une colonne	« séparateurs identiques comme unique » cochée	décocher
« L'écriture n'appartient pas à un exercice ! »	format de date j jmmaa	j jmmaaaa
« La longueur du compte est incorrecte ! »	longueur des comptes tiers du dossier < 17	recréer le dossier avec 17
Section absente après import	l'export envoyait Saaba au lieu de SAAB	section analytique sur 4 caractères (fait)
Pièce refusée pour déséquilibre	ligne de caisse globale retirée	gardée (fait)
Ligne rejetée sur un tiers	tiers créé dans l'application, absent de Sage	créer le tiers dans Sage avant l'import

Points encore ouverts : la longueur maximale réelle du **libellé** dans Sage i7 (35 caractères dans l'ancien modèle de données ; l'application envoie jusqu'à 60). À mesurer dans Sage (taper une longue phrase dans la colonne Libellé de la saisie), puis couper à l'export si besoin, en gardant le libellé complet dans l'application.

10.4 La boucle d'export (version avec lots, à partir du déploiement des correctifs du 08/10)

1. **Contrôles** : plus aucune pièce sur 471000, aucune anomalie bloquante.
2. **Tiers nouveaux** : vérifier que les tiers créés par les caissières depuis le dernier export existent dans Sage ; les créer sinon.
3. **Export Sage** : choisir la période et le journal, lire le bandeau (pièces, lignes, total au débit) et **noter les trois chiffres**.
4. **Créer le lot d'export** et confirmer. Les pièces passent « exportée » et le fichier est conservé en base.
5. Télécharger le **Fichier Sage (.txt)** du lot. Ne l'ouvrez qu'au Bloc-notes, **jamais avec Excel** (Excel réécrit dates et nombres). La « Version Excel » sert à relire, jamais à importer.

6. **Sauvegarder le dossier Sage** (Fichier > Sauvegarde ou copie du .mae).
7. Sage : Fichier > Importer > Format paramétrable > HAKILI_ECRITURES > le fichier. Lire le compte rendu jusqu'au bout.
8. Contrôler : nombre de lignes intégrées, total débit de la balance sur la période, équilibre, numéros de pièces, soldes des tiers, sections.
9. Archiver le fichier et le compte rendu dans un dossier daté.
10. Quand le mois est entièrement dans Sage, le **clôturer** dans l'application (Référentiel > Paramètres > Clôture des mois).

Ne jamais réimporter le même fichier : Sage ne détecte pas les doublons. Si le dossier Sage a été recréé ou restauré et qu'il faut renvoyer des pièces déjà exportées, utilisez « Réexporter des pièces déjà envoyées à Sage » (motif obligatoire, trace en base).

Ne pas lancer « Traitement > Validation des écritures » dans Sage tant que les contrôles ne sont pas faits : avant validation, tout se corrige et se supprime dans Sage.

10.5 La boucle tant que les correctifs du 08/10 ne sont pas déployés

La version en production avant ce déploiement n'a pas de lots : on télécharge le fichier, on l'importe, puis on clique « **Marquer comme exportées** » avec **exactement les mêmes filtres**, immédiatement, en demandant aux validateurs de ne rien valider entre-temps. La case « Déjà exportées » doit rester décochée. Ce fonctionnement est fragile (une pièce validée entre le téléchargement et le marquage passe « exportée » sans être dans le fichier) : c'est une raison de plus de déployer les correctifs.

10.6 Après l'export : corriger une erreur

Une pièce exportée ne se modifie plus, ni dans l'application, ni en base. On passe une **écriture d'extourne** (l'écriture inverse) puis la bonne écriture, dans l'application (qui les exportera) et donc dans Sage. Si une erreur est trouvée dans Sage avant sa validation, on peut supprimer le lot importé dans Sage, corriger la cause, et réimporter le **même fichier de lot** (retéléchargeable à l'identique).

10.7 Comptes et tiers : garder l'application et Sage alignés

- Le plan comptable et le plan tiers de l'application ont été chargés depuis Sage le 18/09/2026 (1 040 comptes, 1 454 tiers). 33 tiers douteux ont été volontairement écartés (intitulés tronqués par l'export Sage, collectifs 409100/419100 absents du plan, codes de brouillard absents de Sage) : à trancher avec le comptable.
- Un compte ou un tiers créé dans l'application doit être créé aussi dans Sage, avec le même code.
- Les comptes collectifs 409100 (avances fournisseurs) et 419100 (avances clients) n'existent pas dans le plan de l'application.

11. Le serveur : se connecter, déployer, sauvegarder

11.1 Ce qu'il y a sur le serveur

Élément	Valeur
Adresse	167.233.234.219 (Linux)
Compte	imane (pas de droits sudo)
Nom d'hôte	guichet-entrepreneur
Dossier du projet	~/hakili_gestion_caisse (un clone Git de la branche main)
Conteneur app	l'application Shiny, publiée sur 127.0.0.1:8022
Conteneur db	PostgreSQL 16, publié sur 127.0.0.1:5432 (jamais sur Internet)
Volume des données	hakili_gestion_caisse_pgdata (volume Docker externe)
Base / utilisateur	Hakili_compta / hakili_admin
Configuration	~/hakili_gestion_caisse/.env (jamais dans Git)
Reverse proxy	nginx de l'hôte , partagé entre toutes les applications, géré par l'administrateur

Le serveur héberge **une douzaine d'autres applications**. Le port 8000 appartient à « guichet-entrepreneur » : c'est pour ça que Hakili tourne sur **8022**. Ne touchez à rien qui n'est pas dans ~/hakili_gestion_caisse.

Le compte imane n'a pas sudo : on ne peut ni installer de paquet, ni modifier nginx, ni poser un certificat. **Tout passe par Docker**. psql et pg_dump ne sont pas installés sur l'hôte : on les utilise à l'intérieur du conteneur db.

11.2 Se connecter

Depuis Windows (PowerShell) ou Linux/macOS :

```
ssh imane@167.233.234.219
cd ~/hakili_gestion_caisse
```

Le mot de passe est remis en main propre (chapitre 0). Recommandations, dans l'ordre :

1. demander à l'administrateur un **compte personnel** pour le repreneur, membre du groupe docker, plutôt que de partager imane ;
2. passer à une **clé SSH** (ssh-keygen -t ed25519, puis ajout de la clé publique dans ~/.ssh/authorized_keys) ;
3. changer le mot de passe de imane après la passation (il est apparu en clair dans une session de travail le 18/09/2026).

Premiers gestes pour vérifier que tout va bien :

```
docker compose ps          # app et db "running" / "healthy"
docker compose logs --tail=50 app # pas d'erreur, "Application startup complete"
git log -1 --oneline        # quel commit tourne
git status                 # doit être propre
```

11.3 Le fichier .env du serveur

Il contient au minimum :

```

POSTGRES_USER=hakili_admin
POSTGRES_PASSWORD=...      # mot de passe de la base
POSTGRES_DB=hakili_compta
ANTHROPIC_API_KEY=...      # clé de l'assistant IA

```

docker-compose.yml construit lui-même DATABASE_URL pour l'application (hôte db, le nom du service dans le réseau Docker). Les autres variables possibles sont dans .env.example (HAKILI_MODELE_IA, HAKILI_BACKUP_RCLONE_REMOTE, etc.).

Ne copiez jamais ce fichier hors du serveur, ne l'affichez pas dans un terminal partagé.

11.4 Déployer une nouvelle version

Principe : **on déploie du code, jamais des données**. La base vit dans le volume ; Git ne contient aucune donnée.

Sur le poste de développement :

```

python verifier.py          # tous les tests sur une base jetable
git add -A
git status                  # uniquement du code, du SQL, des tests
(aucun .xlsx, .env, .log)
git commit -m "Message clair en français"
git push origin main

```

Sur le serveur :

```

cd ~/hakili_gestion_caisse
git status                  # propre, sinon s'arrêter et comprendre pourquoi
# 1. sauvegarde avant tout
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc' \
    > ~/sauvegarde_avant_maj_$(date +%F).dump
ls -lh ~/sauvegarde_avant_maj_*.dump      # le fichier n'est pas vide
# 2. état de la base
docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" <<'SQL'
SELECT count(*) AS lignes, count(DISTINCT id_piece) AS pieces FROM ecritures;
SELECT id_piece FROM ecritures GROUP BY id_piece HAVING round(sum(debit),2) <>
round(sum(credit),2);
SQL
# 3. code
git pull origin main
docker compose build app
docker compose up -d app
# 4. vérification
docker compose logs --tail=80 app        # migrations appliquées, "Application startup complete"

```

Puis recompter les lignes et les pièces (identiques à avant), ouvrir l'application dans un navigateur et se connecter.

Les **migrations se jouent toutes seules** au démarrage du conteneur app. Si l'une échoue, le conteneur ne démarre pas et le journal nomme le fichier : corriger, recommiter, redéployer. Le conteneur db n'a pas besoin d'être recréé.

Retour arrière :

```

git log --oneline -5
git checkout <commit_precedent> -- .
docker compose build app && docker compose up -d app

```

Une migration ne se défait pas toute seule. Si elle a abîmé des données (ça ne devrait jamais arriver, elles sont faites pour ne rien supprimer), restaurer la sauvegarde faite juste avant (11.6).

11.5 Les sauvegardes

État au 08/10/2026 : aucune sauvegarde automatique ne tourne en production. L'ancien script (cron_sauvegarde_host.sh) exige pg_dump sur l'hôte, qui n'y est pas. Le nouveau script backup/sauvegarde.sh passe par le conteneur et ne demande aucun droit administrateur. Il est écrit et testé, mais pas encore déployé ni planifié. **C'est la première chose à faire.**

Une fois le code déployé :

```
cd ~/hakili_gestion_caisse
chmod +x backup/*.sh
./backup/sauvegarde.sh                # premier essai à la main
ls -lh backup/sauvegardes/           # un fichier hakili_AAAAMMJJ_HHMM.dump non
vide                                  # le crontab personnel ne demande pas sudo
crontab -e
```

Ligne à ajouter dans le crontab :

```
30 22 * * * ~/hakili_gestion_caisse/backup/sauvegarde.sh >>
~/hakili_gestion_caisse/backup/sauvegardes/journal.log 2>&1
```

Le script garde 30 jours de sauvegardes (HAKILI_BACKUP_RETENTION_JOURS) et vérifie que chaque fichier n'est pas vide.

Copie hors du serveur : une sauvegarde sur le même disque que la base ne protège ni d'une panne de disque ni d'un piratage. Deux possibilités : - rclone configuré vers un Drive, et HAKILI_BACKUP_RCLONE_REMOTE dans .env (le script copie alors chaque sauvegarde) ; - à défaut, récupérer régulièrement les dumps sur un autre poste : scp imane@167.233.234.219:~/hakili_gestion_caisse/backup/sauvegardes/hakili_*.dump .

Les dumps contiennent toute la comptabilité et les noms des élèves : rangez-les comme des documents confidentiels.

11.6 Restaurer

Essai de restauration (à faire une fois tout de suite, puis chaque trimestre ; ne touche jamais la production) :

```
./backup/restauration_essai.sh          # dernier dump
./backup/restauration_essai.sh backup/sauvegardes/hakili_20261008_2230.dump
```

Le script restaure dans une base temporaire, compare le nombre de pièces et de tiers avec la production, puis supprime la base temporaire.

Restauration réelle (sinistre seulement, après réflexion, application arrêtée) :

```
docker compose stop app
docker compose exec -T db sh -c 'pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean
--if-exists' \
    < chemin/vers/le_dump.dump
docker compose start app
```

Tout ce qui a été saisi après la sauvegarde est perdu : prévenir les centres pour qu'ils ressaisissent.

11.7 Gestes d'administration courants

Besoin	Commande (dans ~/hakili_gestion_caisse)
Redémarrer l'application	<code>docker compose restart app</code>
Voir les journaux en direct	<code>docker compose logs -f app</code>
Ouvrir la base	<code>docker compose exec db psql -U hakili_admin -d Hakili_compta</code>
Plus personne ne connaît le code du comptable	<code>docker compose exec app python outils/definir_code.py comptable</code>
Lister les codes tiers trop longs	<code>docker compose exec -T db psql -U hakili_admin -d Hakili_compta -f - < outils/tiers_trop_longes.sql</code>
Copier un script SQL sur le serveur sans passer par Git	<code>scp fichier.sql imane@167.233.234.219:~/</code> puis l'effacer après usage
Espace disque	<code>df -h ~</code> et <code>du -sh backup/sauvegardes</code>

Changer le mot de passe de la base demande trois gestes solidaires, dans cet ordre, sinon l'application tombe au prochain redémarrage :

1. `ALTER USER hakili_admin WITH PASSWORD '...';` (dans psql) ;
2. mettre le même mot de passe dans `POSTGRES_PASSWORD` du `.env` ;
3. `docker compose up -d --force-recreate app`.

Modifier `POSTGRES_PASSWORD` seul ne change rien : cette variable ne sert qu'à la création d'un volume vide.

11.8 Ce qui demande l'administrateur du serveur

Le compte `imane` ne peut pas le faire ; il faut passer par l'administrateur de la machine (contact à compléter au chapitre « Avant de commencer ») :

- **HTTPS** : poser un certificat (Let's Encrypt) et configurer le `nginx` de l'hôte pour Hakili. Sans HTTPS, les codes d'accès et toutes les données circulent en clair. Au dernier contrôle, l'état n'a pas pu être vérifié : ouvrez l'adresse de l'application dans un navigateur et regardez s'il y a un cadenas. `nginx/hakili.conf` du dépôt est un bon modèle à lui transmettre (il pointe sur `127.0.0.1:8022` et contient les en-têtes `websocket` indispensables à Shiny : sans eux, la page se charge mais reste figée) ;
- les mises à jour du système et de Docker ;
- un compte SSH pour le repreneur ;
- si un jour on veut plusieurs processus Shiny : des sessions « collantes » dans `nginx`.

Afiya avait demandé qu'on lui rappelle ces trois points (certificat, `nginx`, mises à jour système) en fin de projet : ils sont donc toujours ouverts.

11.9 Ne jamais faire en production

- `docker compose down -v` (le `-v` supprime les volumes ; le volume externe protège en principe, mais ne tentez pas le diable) ;
- supprimer le volume `hakili_gestion_caisse_pgdata` ;
- jouer `schema.sql`, `seed.sql`, `production_reset.sql`, `purge_avant_mise_en_service.sql`, `nettoyer_operations.py` ou `import_historique.py` sur `Hakili_compta` ;

- modifier une migration déjà jouée ;
- publier PostgreSQL sur 0.0.0.0 ou l'application hors de 127.0.0.1 ;
- utiliser le port 8000 ;
- `git add -A` sans relire `git status` (classeurs Excel, .env, dumps) ;
- éditer du code directement sur le serveur : la modification serait écrasée au prochain `git pull`, ou bloquerait le `pull`.

11.10 Incidents passés, pour ne pas les revivre

Date	Incident	Leçon
15/09	Une IA tierce avait réécrit le dépôt : port 8000, compose sans base. La production avait été corrigée à la main sans commit	le dépôt doit décrire exactement la production (<code>docker-compose.yml</code> est la référence)
15/09	Quatre migrations jamais jouées en production : « relation ... does not exist »	les migrations sont maintenant jouées automatiquement au démarrage
15/09	PostgreSQL exposé sur Internet (0.0.0.0:5432)	publication limitée à 127.0.0.1
15/09	Assistant en panne en conteneur : <code>chatlas</code> sans son extra <code>anthropic</code>	<code>chatlas[anthropic]</code> dans <code>requirements.txt</code>
18/09	« La liste déroulante des élèves n'apparaît pas » : la base de production n'avait que 4 tiers	avant de chercher dans le code, regarder ce que contient la base
18/09	Mot de passe de la base changé à moitié (<code>ALTER USER</code> sans <code>.env</code>)	les trois gestes du 11.7
23/09	Dossier <code>backup/</code> appartenant à root (ancien montage Docker)	<code>docker run --rm -v ~/hakili_gestion_caisse/backup:/b alpine chown -R 1005:1005 /b</code>
18/09	Session qui se grise, saisie bloquée : connexion morte rendue au pool	<code>_est_cassee()</code> , sondage tolérant aux coupures

11.11 Fins de ligne Windows

Git sous Windows convertit en CRLF : un fichier du PC fait un octet de plus par ligne que le même fichier sur le serveur, à contenu identique. Ce n'est pas une différence de code. Pour comparer le PC et le serveur, comparez après `dos2unix` ou avec `git diff --ignore-cr-at-eol`. Ajouter un fichier `.gitattributes` (*`text=auto eol=lf`) réglerait la question une fois pour toutes (voir chantiers).

12. Développer sur votre poste

12.1 Installer

Prérequis : Python 3.14 (3.12+ fonctionne en principe, la production est en 3.14), PostgreSQL 16 (avec pgAdmin sous Windows), Git, un éditeur (VS Code).

```
git clone https://github.com/imaneodg/hakili-lab-comptabilite.git
cd hakili-lab-comptabilite
git checkout tableau-bord-v2          # voir 12.5 : c'est la branche la plus récente
python -m venv .venv
# Windows : .\.venv\Scripts\Activate.ps1   Linux/macOS : source .venv/bin/activate
pip install -r requirements.txt
```

Dans pgAdmin (ou psql), créez trois bases **vides**, encodage UTF8 :

Base	Usage
hakilisso (ou le nom de votre choix)	votre base de travail
hakili_test	effacée et recréeée par <code>verifier.py</code> à chaque lancement
hakili_verif	base de vérification de <code>verifier_donnees.py</code> (facultative)

Sous Windows, forcez **Encoding: UTF8** à la création : la valeur proposée par défaut (WIN1252) provoque des erreurs d'encodage.

Initialisez votre base de travail :

```
psql -d hakilisso -f sql/schema.sql
psql -d hakilisso -f sql/seed.sql
```

Les migrations se joueront au premier lancement.

Copiez `.env.example` en `.env` et remplissez au minimum :

```
DATABASE_URL=postgresql://utilisateur:motdepasse@localhost:5432/hakilisso
ANTHROPIC_API_KEY=sk-ant-...          # seulement pour l'onglet Assistant IA
```

12.2 Lancer

```
shiny run --reload app.py
```

Puis ouvrez `http://127.0.0.1:8000`. Connectez-vous avec le centre **Siège**, l'identifiant **comptable** et le code initial défini dans `sql/seed.sql` ; changez-le aussitôt dans Référentiel > Mon code d'accès. Créez ensuite une caissière pour un centre (Référentiel > Utilisateurs) et saisissez des opérations sous ce compte pour voir les deux points de vue.

Pensez à saisir des soldes d'ouverture (Référentiel > Paramètres), sinon les caisses passent vite en négatif.

Avoir des données réalistes : `python import_historique.py --dry-run` puis `python import_historique.py` importe les brouillards 2026 de Tampouy et Saaba (fichiers Excel locaux, non versionnés, à demander à Afya) dans la base de `DATABASE_URL`. **Uniquement sur une base locale**. C'est ainsi qu'a été construite la base de démonstration du tableau de bord.

Le serveur MCP facultatif : `python -m mcp_server.server`.

On peut aussi lancer l'ensemble avec Docker comme en production : créer d'abord le volume (docker volume create hakili_gestion_caisse_pgdata), mettre POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB dans .env, puis docker compose up -d --build et ouvrir http://127.0.0.1:8022.

12.3 Les tests

Environ 300 tests pytest, qui tournent **contre une vraie base PostgreSQL**. Les pièces de test sont datées de 2098-2099 pour ne jamais se mélanger à de vraies données, et conf test.py fixe pour elles la date du jour au 01/01/2100.

La façon sûre de tout lancer :

```
python verifier.py
```

Ce script recrée hakili_test depuis schema.sql + seed.sql, joue les migrations et lance pytest dessus. Il ne touche jamais votre base de travail ni la production. Au 08/10/2026 : **301 tests, tous réussis**.

Pour un seul fichier : python -m pytest tests/test_modeles.py -v (attention, il utilise alors le DATABASE_URL de votre .env).

Fichier de test	Ce qu'il protège
test_modeles.py	construction des écritures
test_libelles_multi_mois.py	libellés des encaissements multi-mois, coupe à 60 caractères
test_reglement_fournisseur.py	répartition par mois des règlements fournisseurs
test_transferts_internes.py	transferts entre centres, rapprochement, motif
test_soldes_centre.py	soldes par centre et banque
test_numerotation_date_centre.py	ordre et continuité des numéros définitifs
test_isolation_centres.py, test_authorisation.py, test_autorisation_exhaustive.py	cloisonnement des centres et droits côté serveur
test_auth.py	codes d'accès, verrouillage
test_reactivite_saisie.py	que le formulaire ne se reconstruit pas pendant la saisie
test_analyse.py	fonctions d'analyse
test_assistant_ia.py, test_assistant_hakili.py	outils et compréhension de l'assistant (sans appel au modèle)
test_tableau_bord.py, test_camp_cibles_forces.py	calculs du tableau de bord
test_correctifs_audit_2026_10_08.py	lots d'export, clôtures, protection, dates, codes, 17 caractères...
test_utils.py	petites fonctions
test_ocr_client.py	vide (OCR abandonné)

Règle de travail : **tout correctif vient avec un test** qui échoue avant et passe après. C'est ce qui a permis de modifier un code de cette taille sans régression.

12.4 Conventions de code

- Code, commentaires, messages et commits **en français**. Les commentaires expliquent **pourquoi**, souvent avec la date et l'audit d'origine (« corrigé le 22/09/2026, M3 de l'audit du 18/09 »). Gardez cette habitude : c'est ce qui rend le code compréhensible des mois plus tard.
- Pas d'accents dans les identifiants Python ni dans les commentaires de code (héritage) ; accents dans tout ce que voit l'utilisateur.
- Jamais un code de centre à l'écran : toujours `d.l.nom_centre(ref, code)`.
- Montants : `d.l.fcfa()` (espace comme séparateur de milliers, pas de décimale).
- Requêtes SQL toujours paramétrées.
- Un changement de présentation ne touche ni la logique métier, ni les `id` des champs, ni les appels serveur.
- Garder le code simple : pas d'abstraction tant qu'un seul endroit en a besoin.

12.5 Git : l'état des branches au 08/10/2026

Branche	Contenu
main (local et GitHub)	jusqu'au commit <code>a7236c9</code> (25/09 : règlement fournisseur réparti par mois, validation en lot). C'est ce que la production suit
tableau-bord-v2 (local et GitHub, identiques)	main + 6 commits du tableau de bord, du camp de vacances et du motif des transferts (dernier : <code>4948fc5</code> , 07/10)
copie de travail d'Afiya	tableau-bord-v2 + les correctifs de l'audit du 07/10, non commités (20 fichiers, dont 2 migrations, 2 scripts de sauvegarde, <code>utils/definir_code.py</code> , un fichier de test)
sauvegarde-tableau-bord-v1, origin/sauvegarde-ancienne-version	archives, ne pas utiliser

Le chapitre 13 décrit comment remettre tout cela d'aplomb.

12.6 Le dépôt est public

Au moment de la rédaction, le dépôt GitHub est **public**. Aucune donnée personnelle ni aucun secret n'y est (le `.gitignore` y veille), mais l'adresse du serveur et le code initial du comptable dans `seed.sql` y figurent.

Recommandation : le passer en **privé** (Settings > General > Change visibility), puis ajouter le repreneur comme collaborateur.

13. État au 8 octobre 2026 et chantiers à mener

13.1 Où en est le projet

En production (serveur 167.233.234.219) : - l'application tourne depuis le 15/09/2026 ; dernier déploiement documenté : commit aa7028e (assistant IA reconstruit), le 23/09/2026. main sur GitHub est un peu plus loin (a7236c9, 25/09) : vérifiez sur le serveur avec `git log -1 --oneline` si ce dernier commit a été déployé ; - la base contient le plan comptable (1 040 comptes) et le plan tiers (1 454 tiers) chargés depuis Sage le 18/09, et les pièces saisies depuis la mise en service (base nettoyée le 23/09 pour ne garder que les 39 pièces de référence du 01 au 04/04/2026, puis la saisie réelle). Depuis le 07/10/2026, **toutes les saisies doivent se faire dans l'application** (plus de brouillards Excel) ; - aucune sauvegarde automatique ne tourne ; - l'état du HTTPS n'a pas pu être vérifié.

Prêt mais pas en production : - le **tableau de bord** complet, le **camp de vacances** séparé et le **motif des transferts** : branche `tableau-bord-v2`, poussée sur GitHub, non fusionnée dans main ; - les **correctifs de l'audit du 07/10** (lots d'export, sauvegardes, clôture des mois, protection des pièces exportées, codes tiers à 17 caractères, verrou de validation, rapidité, centre concerné pour le comptable, gestion des codes d'accès, rôle de lecture pour l'assistant, réimport Sage) : dans la copie de travail d'Afiya, **non commités**. 301 tests réussis, parcours vérifiés dans le navigateur, base neuve et base migrée testées.

Dans Sage : le dossier « HAKILI LAB SARL COMPTABILITE DEFINITIF 2026 » a été paramétré le 23/09 (format HAKILI_ECRITURES, sections, dates), mais le dernier blocage noté était la **longueur des comptes tiers** (le dossier devait être recréé avec 17). Le premier import réel n'est pas documenté : **à vérifier avec Afiya ou le comptable** avant tout export.

13.2 À faire en premier (dans l'ordre)

1. Mettre le code au propre et le déployer

Sur le poste qui a la copie de travail d'Afiya (ou en récupérant ses fichiers) :

```
git status                                # 13 fichiers modifiés, 7 nouveaux
python verifier.py                        # 301 tests attendus
git add -A && git status                   # relire : aucun .xlsx, .env, .log, dump
git commit -m "Correctifs de l'audit du 07/10/2026 : lots d'export, sauvegardes, clotures, codes"
git push origin tableau-bord-v2
git checkout main
git merge tableau-bord-v2                 # avance rapide attendue
python verifier.py
git push origin main
```

Puis déployer (chapitre 11.4). Au démarrage, quatre migrations doivent se jouer en production : 2026-10-07_camp_de_vacances, 2026-10-07b_motif_transfert et les deux du 08/10, plus celles du tableau de bord (2026-10-02, 2026-10-03, 2026-10-03b) si elles n'y sont pas encore. Vérifiez dans `schema_migrations`.

Après le déploiement : - si le journal signale « Des codes tiers dépassent 17 caractères » : `utils/tiers_trop_longus.sql`, renommer avant le prochain export, puis rejouer le bloc de contrainte ; - vérifier les pièces saisies par le comptable sur le Siège : `SELECT DISTINCT id_piece, date_piece FROM ecritures WHERE centre = 'SIE'` ; et les corriger vers le bon centre si elles ne sont pas exportées ; - prévenir le comptable que l'export fonctionne désormais par **lots** (chapitre 10.4).

2. Mettre en route les sauvegardes

backup/sauvegarde.sh dans le crontab, une restauration d'essai, une copie hors du serveur (chapitre 11.5 et 11.6). Tant que ce n'est pas fait, toute la comptabilité tient sur un seul disque.

3. Sécurité

- Avec l'administrateur : **HTTPS** sur l'adresse de l'application, compte SSH pour le repreneur.
- Changer le **mot de passe SSH** de imane et le **mot de passe de la base** (ils sont apparus en clair pendant le travail ; le second est aussi dans Claude outputs/env sur le poste d'Afiya, fichier à supprimer).
- Le **code du compte comptable** : le seed crée ce compte avec un code connu, présent dans un dépôt public. Afiya a décidé le 08/10/2026 de le laisser tel quel (seed et production). Le risque est réel tant que le dépôt est public : à rediscuter avec la direction, et au minimum passer le dépôt en privé. Le comptable peut changer son code à tout moment dans Référentiel > Mon code d'accès.
- Passer le **dépôt GitHub en privé**.

4. Vérifier les données de départ

- **Soldes d'ouverture** de chaque caisse (CP et CMD par centre) et de la banque : signalés manquants le 23/09 (« CP Saaba, Pissy, Tampouy sont négatifs sans eux »). Les saisir avec le comptable, à la date de mise en service.
- Les **33 tiers douteux** écartés au chargement du plan tiers : à trancher avec le comptable.
- Le centre « SIA » : vérifier qu'il n'existe pas en double avec un « SIAO » dans la base de production.

5. Trancher les décisions en attente

Sujet	Options	Qui décide
Impôts et taxes (64)	rester hors exploitation, ou passer en charges d'exploitation comme le veut SYSCOHADA (une ligne SQL)	direction + comptable
Réimport vers Sage	la solution livrée le 08/10 (réexport volontaire avec motif, jamais dans un lot) était « encore en discussion »	Afiya / comptable
Accès du directeur	rôle directeur qui voit tableau de bord et assistant en lecture	direction
Longueur du libellé Sage	mesurer dans Sage, couper à l'export si < 60	comptable

13.3 Chantiers, par priorité

Priorité haute (ce mois-ci)

1. **Numéros de pièce sur 4 chiffres**. Le compteur définitif est par journal et par mois, **tous centres confondus**. Avec cinq centres qui saisissent tout dans l'application, la caisse principale peut dépasser 999 pièces dans un mois (rentree). Au-delà, le numéro passe à 4 chiffres et le tri de l'export se dérègle (CP26101000 avant CP2610999). Passer à {n:04d} dans valider_pieces et numero_definitif, après avoir vérifié la longueur du numéro de pièce dans Sage (13 caractères : CP26100001 en fait 10, CMD26100001 11).
2. **Classement du tableau de bord par mot entier** (flux_tableau_bord:~ ('\\m' || motif || '\\M')), et revoir l'ordre « motif sur préfixe court » qui fait passer des vacations en dossiers de conseil (chapitre 8.5). Par une nouvelle migration, avec les chiffres de référence du 8.6 pour contrôler.

3. **Champ « Nature » dans le règlement fournisseur** (vacation, loyer, eau/électricité, internet, entretien, équipement, dossier de conseil, autre), pour ne plus deviner la charge au libellé.
4. **sql/seed.sql** : journal Banque sur 521200 (et compte 521200 « Banques (CBI) »), journal BDU-BF inactif, pour qu'une base neuve ressemble à la production.
5. **resoudre_tiers dans la transaction de l'enregistrement** : aujourd'hui un tiers est créé sur sa propre connexion juste avant l'enregistrement ; si celui-ci échoue, le tiers reste en base, orphelin. Sans gravité, mais à faire proprement.
6. **Journaux Docker** : ne garder que la sortie standard et ajouter une rotation dans docker-compose.yml (logging: {driver: json-file, options: {max-size: "10m", max-file: "5"}}).
7. **Supervision** : une sonde externe gratuite sur l'adresse de l'application, qui prévient si elle tombe.

Priorité moyenne

8. **Formulaires** (demandes du 03/10) : compte 706 dédié au camp (à créer aussi dans Sage) ; avance au personnel séparée du salaire (421000 plutôt que 422300) ; 632800 trop large dans « Dépense courante » (transport/parking, droits et quittances à séparer, avec le comptable).
9. **Tableau de bord** : choix « pièces validées seulement » pour les chiffres présentés à la direction ; version directeur ; prévision de trésorerie sur 3 mois ; corriger le texte du contrôle « Dates aberrantes ».
10. **Interface** : accents manquants dans le formulaire de saisie et les notifications (« MODELE D'OPERATION », « ELEVE »...), montants préremplis « 0.0 », texte anglais de la grille (« Viewing rows... »), colonne Libellé du Brouillard qui montre une ligne quelconque de la pièce (trier les lignes dans table_pieces).
11. **Intégration continue** : une action GitHub avec un service PostgreSQL qui lance la suite de tests à chaque push.
12. **Image Docker** : utilisateur non root, pytest et httpx dans un requirements-dev.txt séparé.
13. **Performance** : si l'application ralentit (un seul processus), déporter le tableau de bord avec reactive.extended_task, ou plusieurs workers avec sessions collantes côté nginx.

Ménage

14. Supprimer les fichiers vides (demo_manuel.py, mock_ocr_server.py, services/ocr_client.py, tests/test_ocr_client.py), nettoyer_operations.py, backup/donnees_avant_correctifs_sage_2026-09-23.py, backup/pg_backup.py et backup/cron_sauvegarde_host.sh une fois sauvegarde.sh en place.
15. Ajouter un .gitattributes (* text=auto eol=lf).
16. Remplacer le README.md par un court texte qui renvoie à docs/.
17. Découper app.py (~3 750 lignes) en modules par onglet (saisie, brouillard, validation, export, contrôles, référentiel), comme l'a été le tableau de bord. À faire avec les tests de réactivité et d'autorisation comme filet.

Idées abandonnées ou en sommeil

- **OCR des factures** : un projet de lecture automatique des factures pour préremplir les formulaires (consignes détaillées dans le document « prompt » du projet claude.ai). Un client et un faux serveur avaient été écrits puis vidés ; rien n'est actif. À reprendre seulement si la direction le redemande, en respectant le principe : préremplir, jamais enregistrer sans contrôle humain.
- **Pièces justificatives numériques** (photo du reçu attachée à la pièce) : demandé au début, jamais commencé.

- **Import des brouillards historiques en production** : écarté le 16/09/2026. Si la direction change d'avis, la liste des corrections à faire d'abord sur les fichiers est dans qualite-donnees-et-formulaires-2026-10-03.md (projet claud.ai) : dates fausses, soldes d'ouverture, reports, approvisionnements qui ne collent pas, motifs des remises au SIAO, comptes à contre-emploi, camp, tiers, pages manquantes.

13.4 Ce qui a été vérifié et qui tient

Pour savoir sur quoi s'appuyer sans tout revérifier (audit du 07/10/2026) :

- installation neuve et migrations complètes et rejouables ;
- équilibre par pièce, signe des montants, unicité des numéros et des rapprochements de transferts, imposés par la base ;
- cloisonnement des centres sur toutes les actions, refusé côté serveur ;
- correction d'une pièce en une seule transaction, avec verrou ;
- une pièce renvoyée puis revalidée reprend son numéro ;
- tableau de bord réservé au comptable côté serveur, cibles enregistrées en une transaction.

14. Historique et décisions

Ce chapitre explique **pourquoi** le code est comme il est. Avant de « simplifier » quelque chose qui paraît compliqué, cherchez ici : c'est souvent la correction d'un vrai problème.

14.1 Chronologie

Date (2026)	Étape
Août	Première version en R sur classeurs Excel, puis portage en Python (Shiny for Python) sur PostgreSQL
05/09	Codes d'accès hachés bcrypt, verrouillage après 5 échecs, trace des suppressions, versions figées, Docker
10/09	Audit complet : droits refaits côté serveur sur dix actions ; écran instable corrigé (onglets construits une fois par connexion, <code>reactive.isolate</code> , montants envoyés en quittant le champ) ; export en cp1252
11/09	Refonte visuelle (barre latérale marine, cartes blanches, bleu #1A5FD0) ; une caisse physique par centre avec ses propres soldes ; changement de banque BDU-BF → CBI (521200), l'ancien journal gardé inactif ; validateur local limité à son centre
12/09	Transferts entre centres par le compte 585000 avec référence de rapprochement ; la validation dit ce qu'elle a écarté et pourquoi
15/09	Remise en service : dépôt réaligné sur la production (port 8022, service <code>db</code>), migrations rattrapées, PostgreSQL refermé sur 127.0.0.1, <code>chatlas[anthropic]</code> ; une pièce exportée ne peut plus être renvoyée ; « exportée » seulement si validée ; double clic neutralisé
16/09	Décision : pas d'import des brouillards historiques, la production démarre vide
18/09	Plans comptable (1 040) et tiers (1 454) chargés depuis Sage ; session grisée corrigée (connexions mortes, sondage tolérant) ; saisie multi-mois ; filtre centre en validation
22/09	Correctifs de l'audit du 18/09 : équilibre et signe imposés par la base, correction de pièce atomique, migrations jouées au démarrage, double rapprochement des transferts impossible
23/09	Premiers essais d'import Sage (CRLF, sections sur 4 caractères, pièces équilibrées) ; création de comptes et de tiers ouverte à tous ; compte d'attente bloquant ; correction de toute pièce non validée ; nettoyage de la base de production ; virements internes à part dans le Brouillard
24-25/09	Assistant IA reconstruit (outils dans l'application, compréhension des fautes, une seule définition des chiffres), puis réorienté vers le directeur ; numérotation par date puis centre, numéro rendu à une pièce renvoyée ; règlement fournisseur réparti par mois ; « Tout sélectionner » en validation
02-03/10	Tableau de bord : d'abord trois chiffres et compte de résultat mensuel, puis version direction (vue d'ensemble + fiche centre), aux couleurs de l'application
07/10	Camp de vacances séparé ; cibles par centre ; motif obligatoire des transferts ; toutes les saisies passent désormais par l'application ; audit complet
08/10	Correctifs de l'audit du 07/10 (non déployés) ; rédaction de ce dossier

14.2 Les décisions métier

Décision	Date	Pourquoi
Sage reste le livre officiel ; l'application prépare et contrôle	dès le départ	Hakili Lab a déjà son dossier Sage et son comptable

Décision	Date	Pourquoi
La caissière ne voit jamais « débit » ni « crédit »	dès le départ	les caissières ne sont pas comptables ; les modèles traduisent
Chaque centre a sa caisse et ses soldes d'ouverture ; la banque est commune	10-11/09	confirmé par le comptable ; un solde commun masquait une caisse à sec
Pas d'import des brouillards historiques	16/09	trop d'erreurs dans les fichiers ; on repart propre
Création de comptes et de tiers ouverte à tous les rôles, tiers actif immédiatement	23/09	une caissière ne doit pas être bloquée ; le format est contrôlé à la création
Le compte d'attente 471000 bloque la validation	23/09	il n'existe pas dans Sage
Libellés : nature et mois d'abord, nom ensuite ; mois abrégés et tous cités (OCT - NOV - DEC)	18 et 25/09	le comptable veut lire chaque mois ; le code tiers identifie déjà la personne
Libellé envoyé entier à Sage (60 caractères), pas de coupe à 35	23/09	demande d'Afiya, en attendant de mesurer la limite réelle de Sage
Numéro définitif à la validation, par date puis centre (SIAO, Tampouy, Saaba, Pissy, Nagrin)	25/09	les numéros se suivent dans Sage, qui lit par date
Ne pas donner le rôle validation aux centres	25/09	le comptable valide seul
L'assistant parle au directeur, en mots simples	24/09	c'est lui qui pose les questions
Tableau de bord en base caisse ; Résultat = argent reçu – charges d'exploitation payées ; caisses et banque séparées	02/10	même logique que l'analyse de Tampouy qui a servi de modèle
Le Siège n'est pas un centre ; la contribution au siège = contribution au SIAO	02/10	règle du propriétaire
Camp reconnu au libellé, jamais au mois	03/10	un cours de juillet reste un cours
Achats d'équipement hors exploitation	03/10	un achat de matériel n'est pas une dépense courante
Motif obligatoire et sans valeur par défaut pour les transferts	07/10	un choix prérempli est validé sans être lu ; un prêt était compté comme une charge
Toutes les saisies dans l'application, plus de brouillards	07/10	harmoniser les libellés, fiabiliser les chiffres
Comptable : centre concerné obligatoire	08/10	ses saisies portaient au Siège
Date d'opération au plus 7 jours après aujourd'hui	08/10	une faute de frappe (2062) ne doit pas passer
Écrans de travail : année en cours + pièces non exportées	08/10	rapidité ; l'historique se relit par période
Assistant = mêmes chiffres que le tableau de bord pour résultat, charges, marge	08/10	jamais deux réponses différentes à la direction
Code du comptable laissé tel quel	08/10	décision d'Afiya, voir chantiers
Impôts (64) en exploitation	reporté	à trancher avec la direction
Une pièce exportée ne peut être réexportée qu'avec un consentement explicite	08/10	éviter les doublons dans Sage

14.3 Les décisions techniques

Décision	Pourquoi
PostgreSQL plutôt qu'Excel ; contraintes et triggers en base	plusieurs postes en même temps, et aucune erreur de code ne doit pouvoir déséquilibrer la comptabilité
Numérotation par compteur <code>INSERT ... ON CONFLICT ... RETURNING</code>	atomique sans verrou applicatif
Validation sous <code>SELECT ... FOR UPDATE</code>	deux validateurs simultanés perdaient des numéros
Sondage d'une ligne <code>revision</code> toutes les 2 s	voir les saisies des autres postes sans tout relire
Révisions séparées écritures / référentiel ; connexion sans effet	une simple connexion relançait tous les calculs de toutes les sessions
Onglets construits une fois par connexion	une saisie ailleurs effaçait le formulaire en cours
Migrations jouées au démarrage, échec = pas de démarrage	la panne du 15/09 venait de migrations oubliées
Lots d'export qui figent pièces + fichier + statut	le marquage séparé faisait sortir des pièces sans qu'elles entrent dans Sage
Trigger de protection des pièces exportées et des mois clôturés	seules les règles Python protégeaient le passé
Contrôles vectorisés, seules les pièces non exportées	54 s → moins d'une seconde à 25 000 pièces
Outils de l'assistant dans l'application (plus de sous-processus MCP)	démarrage fragile, secrets transmis, réponses sans outils
Requête libre de l'assistant sous un rôle PostgreSQL de lecture	le filtre par nom de table était contournable
Sauvegarde par <code>docker compose exec db pg_dump</code>	pas de <code>pg_dump</code> ni de <code>sudo</code> sur l'hôte
Icônes et polices servies localement	les postes de caisse peuvent perdre Internet

14.4 Les audits

Les rapports complets sont dans le projet `claud.ai « tu »` :

Audit	Points principaux	État
10/09	droits côté serveur, écran instable	corrigés
15/09	dépôt ≠ production, migrations, PostgreSQL exposé	corrigés
18/09	équilibre en base, correction non atomique, migrations, export, sauvegardes, HTTPS	corrigés sauf HTTPS (administrateur)
22/09 et 25/09	suivi, dette technique	intégrés
07/10	sauvegardes, export, code 9999, codes tiers, performance, numéros perdus, clôture, dates, transferts, assistant ≠ tableau de bord, Siège, SQL de l'assistant, codes d'accès	corrigés le 08/10 (non déployés) sauf code 9999 (décision), impôts (reporté), A2/A3/A5, interface, dette

15. Glossaire et questions fréquentes

15.1 Glossaire

Mot	Sens
Brouillard	cahier de caisse : liste chronologique des opérations. Avant, un classeur Excel par caisse ; aujourd'hui, l'onglet du même nom
CP / CMD	caisse principale / caisse menues dépenses, deux caisses physiques par centre
Approvisionnement (APPROV CMD)	passage d'argent de la CP vers la CMD
Pièce	une opération enregistrée (plusieurs lignes équilibrées)
Pièces liées	deux pièces d'une même opération sur deux journaux (approvisionnement, versement en banque)
Numéro provisoire / définitif	donné à la saisie (PIS - 2610 - 012) / à la validation (CP2610003)
Journal	regroupement des pièces par nature (CP, CMD, Banque, VTE, ACH)
Compte collectif	compte qui regroupe tous les tiers d'un type (411000 élèves, 401000 fournisseurs, 422000 personnel)
Tiers	élève, fournisseur, enseignant, avec un code propre (411KABORE)
Compte d'attente (471000)	compte provisoire quand on ne sait pas encore ; à reclasser avant validation ; n'existe pas dans Sage
Virements de fonds (585000)	compte de passage de l'argent qui change de caisse sans quitter Hakili Lab
Transfert entre centres	remise d'argent d'un centre à un autre (contribution, prêt, remboursement, impôt), saisie par chacun des deux centres
Contribution au SIAO (au siège)	somme envoyée chaque mois par chaque centre au SIAO, jamais remboursée
Section analytique	code du centre dans Sage (PSSY, TAMP...), sur 4 caractères
Lot d'export	ensemble figé de pièces validées et de leur fichier Sage
Réimport	renvoi volontaire, avec motif, de pièces déjà exportées (dossier Sage recréé)
Extourne	écriture inverse qui annule une écriture déjà dans Sage
Clôture d'un mois	le mois ne reçoit plus de pièce et ses pièces ne changent plus
Frais CA / CV	frais de cours d'appui / de camp de vacances
Vacation, état de vacation	rémunération d'un enseignant à l'heure ; l'état de vacation est la facture correspondante (retenue de 2 %)
Retenue à la source	part retenue sur un paiement et reversée à l'État (2 % vacations, 5 % gardiennage, IRF loyers)
Base caisse	compter à la date du paiement (règle du tableau de bord)
Exploitation / hors exploitation	activité courante / emprunts, prêts, équipement, impôts (pour l'instant)
Plafond salarial	salaires maximum pour garder la marge visée
Capital de base	argent à garder en caisse, en mois de charges (2 par défaut)
SYSOCHADA	référentiel comptable des pays de l'OHADA, dont le Burkina Faso

Mot	Sens
Sage 100 (i7)	logiciel comptable officiel de Hakili Lab
HAKILI_ECRITURES	nom du format d'import paramétré dans Sage
Seed	données de départ d'une base neuve (sql/seed.sql)
Migration	script SQL qui fait évoluer une base existante (sql/migrations/)

15.2 Questions fréquentes

Une caissière dit que l'écriture ne se construit pas. Le ruban sous l'écriture dit ce qui manque : un compte non choisi, un montant à zéro, une répartition qui ne fait pas le total, un motif de transfert absent. Si tout semble rempli, cherchez « Echec de construction des lignes » dans les journaux : un modèle a levé une exception.

« **Le mois de ... est clôturé** ». Le comptable a clôturé ce mois. Dater l'opération dans un mois ouvert, ou rouvrir le mois (Référentiel > Paramètres > Clôture des mois) si l'opération doit vraiment y être et que Sage sera corrigé en conséquence.

Une caisse affiche un solde négatif. Soit le solde d'ouverture n'a pas été saisi pour ce centre, soit une entrée manque (souvent un approvisionnement CMD saisi d'un seul côté ou un transfert reçu non saisi), soit un montant est faux.

Une pièce ne veut pas se valider. Le message dit pourquoi : anomalie bloquante (onglet Contrôles), compte 471000 à reclasser, pièce « à corriger » qui attend la caissière, ou déjà validée.

Un élève n'apparaît pas dans la liste. Taper son nom : il sera créé à l'enregistrement. S'il existait l'année précédente, retaper exactement son nom ou son code le réactive. Vérifier ensuite qu'il existe dans Sage avant l'export.

Sage rejette des lignes à l'import. Lire le compte rendu : tiers absent de Sage (le créer), compte absent, date hors exercice, longueur de code. Voir le tableau du chapitre 10.3. Ne jamais importer « seulement les lignes manquantes » à la main : supprimer le lot dans Sage, corriger, réimporter le fichier entier du lot.

L'application ne démarre plus après un déploiement. `docker compose logs --tail=100 app`. Si c'est une migration, le message nomme le fichier et l'erreur PostgreSQL. Corriger par un commit, ou revenir au commit précédent (chapitre 11.4).

L'écran se grise. La session Shiny est tombée (redémarrage du serveur, coupure réseau). Recharger la page. Si ça se répète, regarder les journaux à l'heure indiquée.

L'assistant répond « la clé est refusée » ou « modèle introuvable ». `ANTHROPIC_API_KEY` ou `HAKILI_MODELE_IA` dans le `.env` du serveur, puis `docker compose up -d --force-recreate app`.

Le tableau de bord classe une dépense dans la mauvaise rubrique. Regarder son compte et son libellé, puis la règle de `classement_comptes` qui l'a attrapée. Corriger par une nouvelle migration (chapitre 8.4), jamais en changeant le libellé des pièces exportées.

Plus personne ne connaît le code du comptable. Sur le serveur : `docker compose exec app python outils/definir_code.py comptable`.

Comment ajouter un centre ? Une migration : `INSERT INTO centres` (code de 3 lettres, intitulé, section analytique de 4 caractères, ordre, actif), les lignes de `soldes_ouverture_centre` pour CP et CMD,

parametres_centre, des alias dans centres_alias. Créer la section dans Sage. Puis créer les utilisateurs du centre dans l'application.

Comment ajouter un modèle de saisie ? Chapitre 4.1, avec un test.

Où sont les vraies données pour tester ? Les brouillards 2026 (fichiers Excel locaux, non versionnés) et import_historique.py sur une base locale. Jamais la base de production.